PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO
Pós-Graduação *Inteligência Artificial Generativa & Large Language Models*

CARLOS FREDERICO COELHO LISBOA

AUDITORIA DE PROJETOS POWER BI

Rio de Janeiro
2026

CARLOS FREDERICO COELHO LISBOA

AUDITORIA DE PROJETOS POWER BI

Trabalho apresentado ao curso GenAI e LLM MASTER da Pontifícia Universidade Católica do Rio de Janeiro como pré-requisito para conclusão de curso e obtenção de crédito na disciplina "Projetos de Sistemas Inteligentes de Apoio à Decisão".

Orientadora: Evelyn C. S. Batista

Rio de Janeiro

2026

LISTA DE TABELAS

Tabela 1 Ferramentas de verificação de boas práticas no Power BI 16
Tabela 2 O sinal que cada regra lê e a forma da afirmação 24
Tabela 3 Pilha tecnológica da ferramenta 25
Tabela 4 Desempenho das regras contra o gabarito em P1–P7 (resultado oficial (tag regras--gt)) 32
Tabela 5 Desempenho das regras após a correção da MOD-003 e da PERF-001 (análise de erros) 34
Tabela 6 Recuperação vencedora (fragmento por seção, busca BM25, corpus completo): médias macro por regra em 62 consultas 35
Tabela 7 Pertinência dos trechos complementares por estratégia de busca (média macro) . 36
Tabela 8 Rubrica humana das explicações geradas: proporção de “sim” por item 37
Tabela 9 Rubrica humana das explicações por regra 37
Tabela 10 Concordância entre a leitura humana e a conferência automática (a: ambos sim; b: só o automático diz sim; c: só o humano diz sim; d: ambos não) 38

LISTA DE FIGURAS

Figura 1 Trecho simplificado de um `model.bim` (projeto P1), com uma coluna calculada, uma medida, uma partição em Power Query M e um relacionamento 18
Figura 2 Fluxo da auditoria 25
Figura 3 Instrução de sistema enviada ao modelo de linguagem 28
Figura 4 Achado do P8 com a explicação gerada pela ferramenta (identificador da tabela abreviado) 39

SUMÁRIO

1 Introdução 6
1.1 Contexto 6
1.2 Problema 6
1.3 Objetivos 7
1.4 Contribuições 7
1.5 Organização do texto 8
2 Fundamentação e trabalhos relacionados 9
2.1 O Power BI e o modelo semântico 9
2.1.1 Modos de armazenamento e o VertiPaq 9
2.1.2 Tabelas, colunas e tipos de dados 10
2.1.3 Relacionamentos, cardinalidade e direção do filtro 10
2.1.4 DAX: medidas, colunas calculadas e tabelas calculadas 11
2.1.5 Power Query e a linguagem M 11
2.2 Boas práticas de modelagem 11
2.2.1 O esquema estrela 11
2.2.2 A documentação oficial como referência 12
2.3 Análise estática e o custo dos falsos positivos 12
2.4 Modelos de linguagem e geração aumentada por recuperação 13
2.4.1 Modelos de linguagem 13
2.4.2 Alucinação e fidelidade 13
2.4.3 Geração aumentada por recuperação 14
2.4.4 Recuperação lexical e densa 14
2.5 Ferramentas existentes 15
3 Método e arquitetura 17
3.1 Visão geral 17
3.2 Leitura do projeto PBIP 17
3.3 Critério de detectabilidade 20
3.4 As regras 21
3.4.1 Regras de modelagem 21
3.4.2 Regras de desempenho 23
3.4.3 Regra de DAX 23
3.5 Fluxo da auditoria 24
3.6 Base de documentação e recuperação 26
3.7 Geração com citação obrigatória 27

3.8 Conjunto de avaliação 28
4 Avaliação 30
4.1 Desenho da avaliação 30
4.1.1 Achado, gabarito e os quatro desfechos 30
4.1.2 Precisão e revocação 30
4.1.3 Casos indeterminados: principal, piso e teto 31
4.1.4 Intervalo de confiança de Wilson 31
4.1.5 Concordância e kappa de Cohen 31
4.2 Regras 32
4.3 Recuperação 34
4.4 Geração 36
4.5 Estudo de caso: o projeto P8 38
5 Discussão e ameaças à validade 40
5.1 O que os resultados mostram 40
5.2 Ameaças à validade 41
5.2.1 Validade de construto 41
5.2.2 Validade interna 41
5.2.3 Validade externa 42
5.2.4 Validade de conclusão e reprodutibilidade 43
5.3 Limites de escopo 43
Referências 45

1 INTRODUÇÃO

1.1 CONTEXTO

O Power BI é a plataforma de análise de dados da Microsoft, usada para construir relatórios e painéis a partir de dados de negócio. Por trás de cada relatório há um *modelo semântico*: o conjunto de tabelas, relacionamentos e fórmulas que organiza os dados e que cada gráfico consulta. A correção e o desempenho de tudo o que o relatório mostra dependem desse modelo. Uma fórmula que divide por um valor que pode ser zero, um relacionamento que propaga filtros nos dois sentidos sem necessidade ou uma opção padrão esquecida ligada podem produzir números inesperados, consultas lentas ou um modelo maior do que precisa ser.

Esses modelos são construídos, em boa parte, por analistas de negócio, que dominam os dados e as perguntas da empresa, mas nem sempre conhecem as boas práticas de modelagem que a própria Microsoft documenta. A Microsoft mantém essas boas práticas publicadas no Microsoft Learn, o seu portal de documentação, e existem ferramentas que verificam regras sobre um modelo, como o *Best Practice Analyzer* do Tabular Editor, apresentado no Capítulo 2. Verificar um modelo, porém, é só metade do trabalho: quem recebe um aviso precisa entender por que aquilo é um problema naquele modelo e o que fazer, e a literatura sobre ferramentas de análise mostra que avisos falsos ou mal explicados fazem os usuários abandoná-las (Johnson et al., 2013; Winters; Manshreck; Wright, 2020).

Dois desenvolvimentos recentes tornam possível uma abordagem nova. O primeiro é o formato PBIP, com o qual o Power BI Desktop passou a gravar um projeto como arquivos de texto, em vez de um arquivo binário fechado, o que permite que um programa leia a definição completa do modelo (Microsoft, 2026). O segundo são os modelos de linguagem de grande porte, capazes de redigir explicações em linguagem natural, e a técnica de geração aumentada por recuperação, que entrega ao modelo de linguagem trechos de documentação para que ele responda com base neles (Lewis et al., 2020). Esses modelos, porém, inventam com frequência informações que soam convincentes (Ji et al., 2023), o que é especialmente perigoso numa auditoria.

1.2 PROBLEMA

O problema que este trabalho aborda é como auditar automaticamente um modelo semântico do Power BI de modo que cada problema apontado seja **confiável**, por ser detectado de forma reprodutível e fundamentado na documentação oficial, e **compreensível**, por vir acompanhado de uma explicação e de uma recomendação específicas para aquele modelo, sem que o modelo de linguagem usado para explicar possa inventar problemas ou fundamentos.

A tensão está em combinar as duas qualidades. Regras determinísticas são confiáveis e mensuráveis, mas a sua explicação é um texto fixo, igual para todos os casos. Um modelo de linguagem escreve explicações sob medida, mas pode errar com segurança. O trabalho investiga se a divisão de papéis entre as duas técnicas, com as regras decidindo e o modelo de linguagem apenas explicando, com a obrigação de citar a documentação, entrega as duas qualidades, e mede até onde isso funciona.

1.3 OBJETIVOS

O objetivo geral é desenvolver e avaliar uma ferramenta que audite projetos do Power BI no formato PBIP, combinando regras determinísticas ancoradas na documentação oficial da Microsoft com explicações geradas por um modelo de linguagem executado localmente, com citação obrigatória da documentação.

Os objetivos específicos são:

1. ler o modelo semântico de um projeto PBIP a partir do arquivo `model.bim` e convertê-lo numa representação interna normalizada;
2. definir um critério explícito para decidir quais regras de boas práticas podem ser verificadas de forma confiável a partir do arquivo, e implementar as regras que o satisfazem, cada uma sustentada por uma passagem citável do Microsoft Learn;
3. construir uma base de documentação a partir do Microsoft Learn e escolher, por medição, o método de recuperação dos trechos;
4. gerar, para cada achado, uma explicação e uma recomendação com um modelo de linguagem local, validando por código que a resposta cita a documentação fornecida;
5. oferecer a auditoria por linha de comando, por uma interface gráfica e por um relatório em HTML e PDF;
6. avaliar separadamente a detecção, a recuperação e a geração, contra julgamento humano, em projetos públicos.

A avaliação responde a três perguntas, uma para cada camada da ferramenta, retomadas no Capítulo 4:

- **Pergunta 1**: as regras apontam os problemas certos, com que precisão e revocação?
- **Pergunta 2**: a recuperação encontra a passagem da documentação que sustenta cada achado?
- **Pergunta 3**: a explicação gerada é pertinente, fiel à documentação e tecnicamente correta, a ponto de poder ser entregue sem edição?

1.4 CONTRIBUIÇÕES

O trabalho faz quatro contribuições.

A primeira é um **critério de detectabilidade** para regras de auditoria de modelos, formulado em três testes aplicados antes de escrever qualquer regra: o problema precisa ser

decidível no arquivo, precisa haver uma passagem citável da documentação que o sustente e a evidência no arquivo precisa ser suficiente para afirmar o problema sem confundi-lo com casos parecidos. O critério reduziu o conjunto previsto de 20 a 25 regras para 9, e essa redução é tratada como resultado do método, e não como falta de esforço: a regra de maior rendimento esperado foi rejeitada pelo terceiro teste, com a medição que mostrou por quê.

A segunda é uma **arquitetura** em que o modelo de linguagem não detecta problemas, apenas os explica, e em que a passagem da documentação que sustenta cada regra entra sempre no contexto do modelo, de modo que a qualidade da busca afeta só os trechos complementares.

A terceira é uma **avaliação** das três camadas contra julgamento humano, com regras de decisão fixadas antes das medições e com resultados negativos relatados: as regras atingiram precisão de 0,88, todas as explicações citaram uma passagem pertinente, mas só 52% foram fiéis à documentação e 38% puderam ser usadas sem edição, e a verificação automática de fidelidade mostrou-se incapaz de substituir a leitura humana.

A quarta é a própria **ferramenta**, com o código, os resultados e os gabaritos disponíveis num repositório público, de modo que a avaliação pode ser reproduzida com os mesmos projetos.

1.5 ORGANIZAÇÃO DO TEXTO

O Capítulo 2 apresenta os conceitos sobre os quais o trabalho se apoia: o modelo semântico do Power BI, as boas práticas de modelagem, a análise estática e o custo dos falsos positivos, os modelos de linguagem e a geração aumentada por recuperação, além das ferramentas existentes.

O Capítulo 3 descreve o método e a arquitetura da ferramenta: a leitura do arquivo `model.bim`, o critério de detectabilidade, as nove regras, o fluxo da auditoria, a base de documentação, a geração com citação obrigatória e o conjunto de avaliação.

O Capítulo 4 apresenta a avaliação: as medidas usadas, os resultados das regras, da recuperação e da geração, e o estudo de caso de um modelo de autoria própria.

O Capítulo 5 discute os resultados, as ameaças à validade da avaliação e os limites de escopo da ferramenta.

O Capítulo 6 conclui o trabalho e aponta os trabalhos futuros. Os apêndices trazem o catálogo das regras, a rubrica da avaliação das explicações e as instruções para reproduzir os resultados.

2 FUNDAMENTAÇÃO E TRABALHOS RELACIONADOS

Este capítulo apresenta os conceitos sobre os quais o trabalho se apoia, em quatro frentes. A primeira é o próprio objeto auditado: o modelo semântico do Power BI, com as suas tabelas, relacionamentos e fórmulas. A segunda são as boas práticas de modelagem que as regras verificam. A terceira é a análise estática, a área da engenharia de software que estuda ferramentas que examinam um artefato sem executá-lo, e o que ela aprendeu sobre o custo dos falsos alarmes. A quarta são os modelos de linguagem e a técnica de geração aumentada por recuperação, usados para explicar os achados. O capítulo termina com as ferramentas existentes para auditar modelos do Power BI e com a posição deste trabalho em relação a elas.

2.1 O POWER BI E O MODELO SEMÂNTICO

O Power BI é a plataforma de análise de dados da Microsoft. Nele, um relatório tem duas partes. O *modelo semântico* organiza os dados: define as tabelas, as colunas, os relacionamentos entre as tabelas e as fórmulas de cálculo. O *relatório* é o conjunto de páginas e visuais, como gráficos, tabelas e cartões, que consultam o modelo. Cada visual, ao ser exibido, envia ao modelo uma consulta que filtra, agrupa e resume os dados (Microsoft, 2024a). O modelo é, portanto, a camada da qual depende a correção e o desempenho de tudo o que o relatório mostra, e é a camada que este trabalho audita. O modelo é construído no Power BI Desktop, a aplicação gratuita de autoria, e publicado no serviço do Power BI para ser compartilhado.

2.1.1 Modos de armazenamento e o VertiPaq

Um modelo pode guardar os dados de três modos. No modo de *importação*, o mais comum e o padrão do Power BI Desktop, os dados são copiados das fontes para dentro do modelo. Nesse processo, o mecanismo de armazenamento chamado *VertiPaq* os comprime e os guarda em disco e, quando o modelo é consultado, os carrega inteiros na memória, o que permite respostas muito rápidas. A documentação estima que 10 GB de dados de origem podem ocupar cerca de 1 GB depois da compressão (Microsoft, 2025a). Em contrapartida, os dados do modelo só são tão recentes quanto a última *atualização*, a operação que recarrega os dados das fontes, normalmente agendada. No modo *DirectQuery*, o modelo guarda apenas a estrutura, e cada consulta é traduzida para a linguagem da fonte e enviada a ela; o modo *composto* mistura os dois por tabela. Os projetos avaliados neste trabalho usam o modo de importação, e várias regras, como as de desempenho, tratam de como o VertiPaq armazena os dados.

O VertiPaq armazena cada coluna separadamente, em estruturas próprias. Colunas numéricas são codificadas pelo próprio valor; colunas de texto recebem um dicionário, no qual cada valor distinto ganha um identificador numérico, e a coluna guarda apenas os identificadores (Microsoft, 2024b). Por isso o tamanho do modelo depende não só do número de linhas, mas

também do número de colunas e da quantidade de valores distintos em cada uma: uma coluna de texto com muitos valores distintos ocupa mais espaço que uma coluna numérica.

2.1.2 Tabelas, colunas e tipos de dados

Cada tabela do modelo tem colunas, e cada coluna tem um *tipo de dado* que determina o que ela pode guardar e como é armazenada. Para números, o Power BI oferece três tipos: *Número inteiro*; *Número decimal fixo*, com exatamente quatro casas decimais; e *Número decimal*, armazenado em ponto flutuante, de forma aproximada, com até 15 dígitos de precisão (Microsoft, 2025b). Cada coluna também tem uma *agregação padrão*, a forma como o Power BI a resume quando ela é arrastada para um visual sem nenhuma fórmula: somar, contar, tirar a média ou não resumir.

2.1.3 Relacionamentos, cardinalidade e direção do filtro

Um *relacionamento* liga uma coluna de uma tabela a uma coluna de outra e serve para propagar filtros: quando o leitor do relatório filtra uma tabela, o filtro passa pelos relacionamentos às tabelas relacionadas (Microsoft, 2025c). A documentação dá um exemplo com quatro tabelas, categoria, produto, ano e vendas. Ao filtrar a categoria "Cat-A", o filtro passa à tabela de produtos, isolando os produtos daquela categoria, e daí à de vendas, isolando as vendas desses produtos; um filtro no ano passa diretamente às vendas, e os dois filtros se combinam.

Cada relacionamento tem uma *cardinalidade*, que descreve quantas linhas de cada lado podem se corresponder. No lado "um", a coluna tem valores únicos; no lado "muitos", pode ter valores repetidos. As opções são um-para-muitos (ou, lido ao contrário, muitos-para-um), a mais comum; um-para-um, em que os dois lados têm valores únicos; e muitos-para-muitos (Microsoft, 2025c). Num relacionamento entre produtos e vendas, por exemplo, cada produto aparece uma vez na tabela de produtos e muitas vezes na de vendas: é um relacionamento um--para-muitos.

Cada relacionamento tem também uma *direção do filtro cruzado*. Na direção *única*, o filtro vai sempre do lado "um" para o lado "muitos", dos produtos para as vendas. Na direção *ambas*, chamada de *bidirecional*, ele também volta, das vendas para os produtos. Relacionamentos um-para-um são sempre bidirecionais. A documentação adverte que relacionamentos bidirecionais podem prejudicar o desempenho e criar caminhos ambíguos de propagação, e recomenda usá-los só quando necessário (Microsoft, 2025c). Ela registra ainda que uma tabela sem relacionamentos, dita *desconectada*, é incomum, mas pode ser legítima quando serve para receber uma escolha do leitor, como um valor de câmbio escolhido num seletor e usado depois por uma fórmula (Microsoft, 2025c). Esse caso reaparece na avaliação da regra MOD-003, no Capítulo 4.

2.1.4 DAX: medidas, colunas calculadas e tabelas calculadas

O *DAX* (*Data Analysis Expressions*) é a linguagem de fórmulas dos modelos tabulares, usada no Power BI, no Analysis Services e no Power Pivot do Excel (Microsoft, 2023a). Ela aparece em três tipos de objeto do modelo, que diferem no momento em que são calculados.

Uma *medida* é uma fórmula calculada no momento da consulta, cujo resultado depende do *contexto*: dos filtros aplicados pelo visual, pelos seletores e pelas próprias fórmulas. A medida `Total Sales = SUM(Sales[Sales Amount])`, por exemplo, devolve a soma das vendas de cada categoria quando colocada num visual que agrupa por categoria, e o total geral quando colocada sozinha num cartão. As medidas não guardam valores; são recalculadas a cada consulta (Microsoft, 2023a).

Uma *coluna calculada* é uma coluna acrescentada a uma tabela cujo valor, linha a linha, é definido por uma fórmula DAX, como uma coluna que junta o ano e o trimestre num texto “2017 Q1”. Ao contrário da medida, ela é calculada na atualização dos dados e guardada no modelo, ocupando memória como qualquer outra coluna (Microsoft, 2023a). Uma *tabela calculada* é uma tabela inteira definida por uma fórmula DAX, em vez de carregada de uma fonte; o caso típico é a tabela de calendário gerada pelas funções `CALENDAR` ou `CALENDARAUTO` (Microsoft, 2025d).

2.1.5 Power Query e a linguagem M

Antes de chegar ao modelo, os dados passam pelo *Power Query*, o mecanismo de preparação de dados do Power BI, que se conecta às fontes e aplica transformações como remover colunas, filtrar linhas, mesclar e agrupar consultas (Microsoft, 2026). Cada transformação é registrada como um passo de uma consulta que se repete a cada atualização. Por trás da interface gráfica, toda consulta é escrita na *linguagem M*, a linguagem de transformação do Power Query, que o autor também pode editar diretamente (Microsoft, 2026). Uma mesma coluna derivada pode, assim, ser criada de duas formas: como coluna personalizada no Power Query, em M, ou como coluna calculada no modelo, em DAX. A escolha entre as duas é o assunto da regra PERF-001.

2.2 BOAS PRÁTICAS DE MODELAGEM

2.2.1 O esquema estrela

O *esquema estrela* é a abordagem de modelagem consolidada nos armazéns de dados relacionais e descrita em profundidade por Kimball; Ross (2013). Ela classifica as tabelas em dois papéis. As *tabelas de dimensão* descrevem as entidades do negócio, as “coisas” que se analisam: produtos, clientes, lojas e, quase sempre, o próprio tempo, numa dimensão de data com uma linha por dia. As *tabelas de fatos* registram os eventos ou observações: vendas, pedidos,

saldos de estoque. Uma tabela de fatos tem colunas que apontam para as dimensões e colunas numéricas que são resumidas (Microsoft, 2024a).

A documentação do Power BI explica por que o esquema estrela se encaixa no produto. Cada visual gera uma consulta que filtra, agrupa e resume; as dimensões servem para filtrar e agrupar, e os fatos, para resumir. No modelo, o papel de cada tabela é definido pelos relacionamentos: o lado "um" de um relacionamento um-para-muitos é a dimensão, e o lado "muitos", o fato (Microsoft, 2024a). A documentação recomenda não misturar os dois papéis numa mesma tabela e ter "o número certo de tabelas com os relacionamentos certos", e reconhece que o bom desenho é "parte ciência, parte arte", podendo ser contrariado quando houver razão (Microsoft, 2024a).

Duas variações desse esquema aparecem nas regras. No *floco de neve*, uma dimensão é normalizada em várias tabelas encadeadas, como produto, subcategoria e categoria; em geral, recomenda-se integrá-las numa só tabela. A *dimensão degenerada* é um atributo da própria tabela de fatos usado para filtrar, como o número do pedido, que pode justificar um relacionamento um-para-um (Microsoft, 2024a; 2024c).

2.2.2 A documentação oficial como referência

As boas práticas de modelagem do Power BI estão publicadas na seção de orientação (*guidance*) do Microsoft Learn, o portal de documentação da Microsoft, mantida pela equipe do produto e atualizada com ele. Este trabalho adota essa documentação como a referência normativa das regras, por três razões: ela é pública e pode ser citada; ela traz, junto de cada recomendação, as exceções que a própria Microsoft reconhece; e ela pode ser coletada e indexada para a recuperação, como descrito no Capítulo 3. A escolha tem uma consequência importante para o método: uma regra só é aceita se houver uma passagem da documentação que a sustente, e a explicação gerada para cada achado precisa citar essa passagem.

2.3 ANÁLISE ESTÁTICA E O CUSTO DOS FALSOS POSITIVOS

A *análise estática* é a família de técnicas que examina um artefato de software sem executá-lo, em busca de defeitos ou de desvios de boas práticas. O exemplo mais comum são os verificadores de código-fonte, que apontam erros prováveis enquanto o programador escreve. A ferramenta deste trabalho é uma análise estática no mesmo sentido: examina a definição do modelo semântico, no arquivo, sem carregar os dados nem executar as consultas.

A literatura sobre essas ferramentas registra, de forma consistente, que o maior obstáculo à sua adoção não é a falta de cobertura, mas o excesso de alarmes falsos. Num estudo com vinte desenvolvedores, Johnson et al. (2013) verificaram que, embora todos considerassem as ferramentas úteis, os falsos positivos e a forma como os avisos são apresentados eram barreiras ao uso. A experiência do Google aponta na mesma direção. Sadowski et al. (2018) descrevem uma

infraestrutura de análise estática usada diariamente pela maioria dos engenheiros da empresa, que encontra milhares de problemas por dia corrigidos pelos próprios engenheiros, por escolha deles, e atribuem esse resultado às lições de tentativas anteriores. O critério que a empresa adota é explícito: uma verificação só é mantida se menos de 10% dos seus avisos forem *falsos positivos efetivos*, isto é, avisos sobre os quais o desenvolvedor não age, seja porque estão errados, seja porque não foram entendidos (Winters; Manshreck; Wright, 2020).

Duas lições dessa literatura orientam este trabalho. A primeira é que precisão vale mais que cobertura: uma regra que erra com frequência compromete a confiança no relatório inteiro, e é melhor não ter a regra. É essa a justificativa do critério de detectabilidade descrito no Capítulo 3. A segunda é que a explicação faz parte da precisão: um aviso correto que não é entendido tem, para quem o recebe, o mesmo efeito de um aviso errado. É por isso que a qualidade das explicações geradas é avaliada no Capítulo 4.

2.4 MODELOS DE LINGUAGEM E GERAÇÃO AUMENTADA POR RECUPERAÇÃO

2.4.1 Modelos de linguagem

Um *modelo de linguagem de grande porte* (*large language model*, LLM) é uma rede neural treinada sobre grandes volumes de texto para prever a continuação de um texto. Os modelos atuais usam a arquitetura *Transformer*, baseada num mecanismo de atenção que relaciona cada parte do texto com as demais (Vaswani et al., 2017). O modelo não lê palavras, e sim *tokens*, pedaços de palavra de um vocabulário fixo; a *janela de contexto* é o número máximo de tokens que ele considera de uma vez, somando o pedido e a resposta. Para gerar, o modelo atribui uma probabilidade a cada token possível e escolhe um; a *temperatura* controla essa escolha, de modo que valores baixos favorecem os tokens mais prováveis e tornam a saída mais estável.

Os modelos de *instrução* são versões ajustadas para seguir pedidos em linguagem natural. Alguns são distribuídos abertamente e podem ser executados na própria máquina, como a família Qwen2.5, usada neste trabalho (Qwen Team, 2024). Para caber em equipamentos modestos, esses modelos costumam ser *quantizados*: os seus parâmetros, normalmente guardados com 16 bits, passam a ser guardados com menos bits, como 4, o que reduz a memória necessária com alguma perda de qualidade. A execução local tem uma vantagem própria para a auditoria: o modelo semântico de um cliente, que descreve a estrutura dos seus dados, não precisa ser enviado a um serviço externo.

2.4.2 Alucinação e fidelidade

Os modelos de linguagem produzem, com frequência, texto fluente e convincente que não é sustentado pela informação disponível, fenômeno chamado de *alucinação*. Num levantamento abrangente, Ji et al. (2023) distinguem dois tipos: a alucinação *intrínseca*, em que o texto

gerado contradiz a fonte, e a *extrínseca*, em que o texto não pode ser verificado a partir dela. O conceito oposto é a *fidelidade*: um texto é fiel quando tudo o que afirma é sustentado pela fonte. Para uma ferramenta de auditoria, a alucinação é especialmente perigosa, porque uma explicação errada, escrita com segurança, pode levar o dono do modelo a uma correção errada.

2.4.3 Geração aumentada por recuperação

A *geração aumentada por recuperação* (*retrieval-augmented generation*, RAG) é a técnica de buscar, antes de gerar, trechos de uma base de documentos relevantes para o pedido, e entregá-los ao modelo junto com o pedido, para que ele responda com base neles (Lewis et al., 2020). Lewis et al. (2020), que deram nome à técnica, combinam a memória *paramétrica* do modelo, o que ele aprendeu no treinamento, com uma memória *não paramétrica*, a base de documentos consultada no momento, e relatam respostas mais específicas e factuais que as de um modelo sem recuperação. A técnica tem duas vantagens para este trabalho: a base pode ser a documentação oficial, atualizada, em vez do conhecimento congelado no treinamento do modelo; e a resposta pode ser verificada contra os trechos entregues.

A verificação exige que o modelo diga de onde tirou cada afirmação. Gao et al. (2023) propuseram uma avaliação automática de respostas com citações e mostraram que mesmo os melhores modelos da época deixavam de ter as suas afirmações completamente sustentadas pelas citações em metade das respostas de um dos conjuntos avaliados. Citar, portanto, não garante fidelidade: a citação torna a verificação possível, mas não a dispensa. Essa distinção é medida no Capítulo 4.

2.4.4 Recuperação lexical e densa

A qualidade da geração depende do que a recuperação entrega. Há duas famílias de busca. A *busca lexical* compara as palavras da consulta com as dos documentos. O seu representante clássico, o BM25, dá a cada documento uma pontuação que cresce com o número de vezes que as palavras da consulta aparecem nele, com peso maior para as palavras raras no conjunto de documentos e com um ajuste pelo tamanho do documento (Robertson; Zaragoza, 2009). A *busca densa* converte a consulta e os documentos em vetores numéricos, chamados *embeddings*, produzidos por um modelo treinado para que textos de significado parecido fiquem próximos; os documentos são ordenados pela proximidade com a consulta. Ela encontra documentos que dizem a mesma coisa com outras palavras, mas pode deixar escapar nomes exatos, como o de uma função. A família de modelos de embedding BGE, da qual vem o modelo usado neste trabalho, é um exemplo recente (Xiao et al., 2023).

A busca densa não supera a lexical em todos os casos. Numa comparação sobre dezoito conjuntos de dados de domínios diversos, Thakur et al. (2021) concluíram que o BM25 é uma linha de base robusta quando o sistema é aplicado a um domínio para o qual não foi treinado. As

duas famílias também podem ser combinadas: a *fusão recíproca de posições* soma, para cada documento, o inverso da sua posição em cada uma das listas, e costuma superar cada busca isolada (Cormack; Clarke; Büttcher, 2009). Por essas razões, este trabalho não escolheu a busca de antemão, mas por medição, como descreve o Capítulo 3.

2.5 FERRAMENTAS EXISTENTES

Existem ferramentas consolidadas para verificar boas práticas em modelos do Power BI, e nenhuma delas é substituída por este trabalho. A mais conhecida é o *Best Practice Analyzer* (BPA) do Tabular Editor. O Tabular Editor 2 é um editor de modelos tabulares do Analysis Services e do Power BI gratuito e de código aberto, sob licença MIT, mantido por Daniel Otykier e disponível no GitHub (Otykier, *[S.d.]*); existe também uma versão comercial, o Tabular Editor 3. O BPA verifica o modelo contra um conjunto de regras, lista todos os objetos que violam cada uma, e pode corrigir automaticamente muitos dos problemas que encontra; cada regra é uma condição escrita sobre os objetos do modelo, com uma severidade e uma descrição (Tabular Editor, *[S.d.]*). A Microsoft publica um conjunto de regras para o BPA, organizado em categorias como desempenho, expressões DAX, prevenção de erros, formatação e manutenção (Microsoft, 2023b). Este trabalho usou o BPA apenas como referência conceitual (ADR-004) e escreveu as suas regras do zero, por duas razões: as regras do BPA são condições avaliadas pelo próprio Tabular Editor, o que acrescentaria à ferramenta uma dependência de outro programa, e o critério deste trabalho exige que cada regra seja sustentada por uma passagem da documentação, verificada antes de a regra ser escrita.

No ambiente do Microsoft Fabric, a biblioteca Python *Semantic Link Labs*, publicada pela Microsoft para uso em notebooks, oferece um analisador de boas práticas e um analisador do VertiPaq para modelos semânticos já publicados (Microsoft, 2024d). Para a camada de relatório, o *Fab Inspector*, sucessor do PBI Inspector, testa os arquivos do relatório no formato PBIR com regras declarativas em JSON, que verificam, por exemplo, a qualidade dos visuais (Van Gulck, *[S.d.]*).

Tabela 1 – Ferramentas de verificação de boas práticas no Power BI

Ferramenta	**O que examina**	**Onde executa**	**Como apresenta o achado**
BPA do Tabular Editor	Modelo semântico	Tabular Editor 2 (gratuito, código aberto) ou 3, com o modelo aberto	Regra violada, severidade e descrição da regra; correção automática em muitos casos
Semantic Link Labs	Modelo semântico publicado	Notebook do Microsoft Fabric	Resultado do analisador de boas práticas e do analisador do VertiPaq
Fab Inspector	Camada de relatório (PBIR)	Máquina local, integração contínua ou serviço	Resultado de regras declarativas em JSON
Este trabalho	Modelo semântico no arquivo `model.bim`	Máquina local, sem publicar o modelo e sem acessar os dados	Explicação e recomendação geradas para o achado, com a citação da passagem da documentação que o sustenta

Fonte: Elaborado pelo autor a partir da documentação de cada ferramenta.

A Tabela 1 resume as diferenças. A contribuição deste trabalho não está em verificar mais regras que essas ferramentas, e sim em três características. Cada regra é aceita por um critério explícito, que exige uma passagem citável da documentação e evidência suficiente no arquivo. Cada achado vem acompanhado de uma explicação redigida para o caso concreto, com a citação verificável da passagem que o sustenta. E tanto as regras quanto as explicações foram avaliadas contra julgamento humano, com os resultados publicados: na documentação consultada das ferramentas acima, não se encontrou medição publicada da precisão das suas regras.

3 MÉTODO E ARQUITETURA

Este capítulo descreve a ferramenta desenvolvida e as decisões de método que a sustentam. A apresentação segue o caminho de um projeto auditado: a leitura do arquivo, a detecção dos problemas, a recuperação da documentação que os fundamenta e a redação da explicação pelo modelo de linguagem. As escolhas foram registradas, ao longo do projeto, em registros de decisão de arquitetura (*Architecture Decision Records*, ADR), citados aqui pelo número quando a decisão vem de um deles.

3.1 VISÃO GERAL

A ferramenta recebe um projeto do Power BI Desktop salvo no formato PBIP (*Power BI Project*) e devolve uma lista de achados, cada um com explicação, recomendação e a passagem da documentação oficial que o fundamenta. Ela se organiza em três camadas. A primeira lê o modelo semântico do projeto e o converte numa representação interna normalizada. A segunda aplica regras determinísticas sobre essa representação e produz os achados. A terceira recupera, para cada achado, trechos da documentação do Microsoft Learn e pede a um modelo de linguagem executado localmente que explique o problema e recomende a correção, citando os trechos.

A divisão de responsabilidades entre a segunda e a terceira camadas é a decisão central do projeto (ADR-003): **as regras determinísticas são a única fonte de achados**. O modelo de linguagem não decide se um problema existe; ele apenas explica um problema que uma regra já detectou. Essa separação tem duas consequências. A precisão e a revocação da auditoria passam a depender só das regras, que são reprodutíveis e podem ser medidas contra um gabarito, sem a variação própria da geração. E o modelo de linguagem, cujo erro é mais difícil de controlar, fica restrito à parte em que um erro custa menos: uma explicação ruim degrada a redação, mas não cria nem apaga um achado.

3.2 LEITURA DO PROJETO PBIP

Um relatório do Power BI tem duas partes. O *modelo semântico* reúne as tabelas, as colunas, os relacionamentos entre as tabelas e as medidas de cálculo; é a camada que os visuais consultam. O *relatório* são as páginas e os visuais construídos sobre esse modelo. No formato tradicional, o PBIX, as duas partes ficam dentro de um único arquivo binário, ilegível fora do Power BI Desktop. O formato PBIP (*Power BI Project*) grava o mesmo conteúdo como uma pasta de arquivos de texto, com uma subpasta para o relatório e outra para o modelo semântico, o que permite versionar o projeto e, para o que interessa aqui, lê-lo com um programa.

Na pasta do modelo semântico, o arquivo central é o `model.bim`. Ele contém a definição completa do modelo, escrita em TMSL (*Tabular Model Scripting Language*), a linguagem em

JSON com que a Microsoft descreve os modelos tabulares do Analysis Services, o mesmo motor que executa os modelos do Power BI (Microsoft, 2026). O arquivo guarda a **definição** do modelo, mas não os **dados**: os dados importados ficam numa cópia em cache, num arquivo separado (`cache.abf`) que o Git ignora por padrão (Microsoft, 2026). Por isso a ferramenta audita a estrutura do modelo sem acessar nem os dados nem a fonte de onde eles vêm. Nos projetos deste trabalho, o `model.bim` tem entre 53 KB e 862 KB.

A Figura 1 mostra um trecho simplificado do `model.bim` de um dos projetos avaliados. No primeiro nível ficam o nível de compatibilidade do modelo e o objeto `model`. Dentro dele, a lista `tables` descreve cada tabela: suas colunas, com tipo de dado e forma de agregação padrão, e, no caso das colunas calculadas, a expressão DAX que as define; suas medidas, cada uma com a expressão DAX; e suas partições, com a consulta em Power Query M que carrega os dados. A lista `relationships` descreve cada relacionamento pela tabela e coluna de cada ponta. O arquivo traz ainda hierarquias, papéis de segurança em nível de linha e anotações gravadas pelo Power BI Desktop.

Figura 1 – Trecho simplificado de um `model.bim` (projeto P1), com uma coluna calculada, uma medida, uma partição em Power Query M e um relacionamento

```
{
  "compatibilityLevel": 1606,
  "model": {
    "tables": [
      {
        "name": "Date",
        "columns": [
          { "name": "Date", "dataType": "dateTime", "isKey": true },
          { "name": "Mth", "type": "calculated", "dataType": "string",
            "expression": "FORMAT('Date'[Date],\"MMM\")" }
        ],
        "measures": [
          { "name": "Latest year", "expression": "min('Date'[Year])" }
        ],
        "partitions": [
          { "mode": "import",
            "source": { "type": "m",
                        "expression": ["let", "    Source = Excel.Workbook(...)", "..."] } }
        ]
      }
    ],
    "relationships": [
      { "fromTable": "Sales", "fromColumn": "CustomerKey",
        "toTable": "Customer", "toColumn": "CustomerKey" }
    ]
  }
}
```

Fonte: Elaborado pelo autor a partir do `model.bim` do projeto P1, com propriedades omitidas.

Tudo o que as regras sabem do modelo vem desse arquivo, e duas propriedades dele moldaram o método. A primeira é o que **não** está no arquivo: os valores dos dados e o uso das colunas pelos visuais do relatório. Uma regra que dependa dessas informações não pode ser decidida pela ferramenta, como mostra a próxima seção. A segunda é que o Power BI Desktop omite as propriedades que estão no valor padrão. No relacionamento da Figura 1, não aparecem a cardinalidade nem a direção do filtro porque ambas têm o valor padrão: muitos-para-um, com filtro num só sentido. Nos oito modelos do projeto, a propriedade que torna o filtro bidirecional aparece em apenas 5 dos 57 relacionamentos. Ler um `model.bim` exige, portanto, conhecer o padrão de cada propriedade, e tratar a ausência de uma propriedade como informação só é seguro quando se sabe que ela seria gravada no caso contrário. Essa lição está na origem do terceiro teste do critério de detectabilidade.

A pasta do modelo semântico declara, no arquivo `definition.pbism`, em que formato a definição está gravada: na versão 1.0, obrigatoriamente em TMSL, no `model.bim`; a partir da versão 4.0, em TMSL ou em TMDL (*Tabular Model Definition Language*), um formato mais legível que divide a definição em vários arquivos dentro de uma pasta `definition` (Microsoft, 2026). A ferramenta lê apenas o `model.bim` (ADR-001). O TMSL é JSON, que a biblioteca padrão do Python lê sem dependências, enquanto o TMDL não tinha, à época da decisão, analisador oficial em Python, e salvar em TMDL ainda era recurso em pré-visualização, de conversão irreversível. O formato fica atrás de uma única função de leitura, de modo que acrescentar o TMDL no futuro não exige reescrever as regras. Na prática, basta salvar o projeto com a opção de TMDL desligada para que o Power BI Desktop grave o `model.bim`.

O analisador converte o `model.bim` num modelo interno validado com Pydantic, com tabelas, colunas, medidas, relacionamentos, hierarquias, ordenações, papéis de segurança em nível de linha e as `variations` que ligam uma coluna de data à tabela de data automática criada pelo Power BI. Uma particularidade do TMSL é guardar expressões ora como texto, ora como lista de linhas; o analisador normaliza os dois casos.

As expressões DAX (*Data Analysis Expressions*) não são interpretadas sintaticamente, o que exigiria um analisador completo da linguagem, fora do orçamento do projeto. Elas passam por um *analisador léxico* próprio, um programa que percorre o texto e o divide em unidades mínimas, chamadas *tokens*, sem interpretar a estrutura gramatical da expressão: comentários, cadeias de texto, números, nomes de função, referências a colunas e medidas e operadores. Com os tokens, uma regra pode perguntar, por exemplo, onde há um operador de divisão e o que vem depois dele, sem confundir uma barra dentro de um comentário ou de um texto entre aspas com uma divisão de verdade. A afirmação de que o analisador léxico basta foi medida, não suposta: no modelo do estudo de caso (P8), as 137 expressões DAX foram transformadas em tokens sem

nenhum token desconhecido, e as 105 expressões de autoria do modelador ficaram cobertas pela varredura, sem lacunas. Quando uma expressão não é coberta, a ferramenta declara a lacuna no resultado em vez de silenciar.

3.3 CRITÉRIO DE DETECTABILIDADE

O planejamento inicial previa entre 20 e 25 regras, com uma cota por categoria. A cota foi substituída, ainda no início do projeto, por um critério que pergunta outra coisa: não quantas regras cabem em cada categoria, mas quais problemas podem ser identificados de forma confiável a partir do arquivo. O critério de detectabilidade foi construído em três etapas, cada uma motivada por uma falha concreta, e é aplicado **antes** de escrever o código de uma regra.

O primeiro teste é a **decidibilidade no arquivo**: o problema precisa ser observável no `model.bim`. Ele ordena as regras em grupos de prioridade. As estruturais, que leem propriedades explícitas do JSON, vêm primeiro; as de DAX com padrão textual inequívoco, depois; as de DAX dependentes de contexto de avaliação ficaram fora, porque sem analisador sintático a heurística erra com frequência; e as de Power Query M ficaram fora do produto mínimo.

O segundo teste é a **âncora citável**: uma regra só entra se houver uma passagem do Microsoft Learn que a sustente, lida e transcrita antes do código. Sem essa passagem, a regra não pode cumprir a exigência de que toda explicação cite a documentação. Aplicado ao conjunto estrutural, o teste eliminou três das nove regras propostas. Uma delas marcaria como defeito o uso de tabelas calculadas em DAX, mas a página que a sustentaria recomenda justamente criar tabelas de data com as funções `CALENDAR` e `CALENDARAUTO` (Microsoft, 2025d). As outras duas não tinham passagem que as fundamentasse.

O terceiro teste é a **suficiência da evidência**. Ele nasceu de uma regra que passava nos dois primeiros e ainda assim errava: a MOD-005, que deveria apontar a dimensão de data não marcada como tabela de data, lia a ausência de uma propriedade como prova de que a marcação não fora feita. Mas o Power BI Desktop nunca grava essa propriedade, de modo que a ausência era compatível com os dois casos e a regra marcaria também a dimensão corretamente configurada. O teste exige que a regra enumere por escrito, antes do código, (a) todas as formas de escrever a condição procurada, (b) todas as coisas que se parecem com o sinal sem sê-lo e (c) todos os lugares do arquivo onde o sinal pode estar. Só então a regra declara se afirma pela **presença** ou pela **ausência** de algo; afirmar por ausência exige as três enumerações completas e a garantia de que a presença seria gravada no caso contrário. Faltando qualquer uma, a regra se cala e o falso negativo é declarado. A MOD-005 foi reescrita: passou a usar como prova a `variations` da coluna de data apontando para uma tabela de data automática, que o Power BI remove quando a tabela é marcada. Com o tempo automático desligado, ela se cala.

O caso que melhor ilustra o critério é uma regra rejeitada. A candidata PERF-005 apontaria colunas sem uso no modelo e era a de maior rendimento esperado do grupo, com âncora já transcrita: a página de redução de dados recomenda remover colunas que não sirvam nem ao relatório nem à estrutura do modelo (Microsoft, 2024b). A cláusula (c) a derrubou. O uso de uma coluna pelo relatório mora na camada de visuais, que a ferramenta não lê, e a regra afirmaria uma ausência sobre um domínio que não observa. A medição no P8 confirmou o problema: dos 33 achados que a regra emitiria, 5 eram defensáveis, 3 eram falsos positivos confirmados e os 25 restantes dependiam da camada de relatório. A precisão ficaria entre cerca de 15% e cerca de 91%, conforme o uso dessas 25 colunas, uma faixa larga demais para sustentar qualquer número diante do limiar de 0,7 adotado no projeto. A regra não foi implementada.

Por isso, a contagem final de regras deixou de ser meta e passou a ser **resultado** do método. O conjunto tem 9 regras, menos da metade do previsto, e a diferença não é lacuna de esforço: cada regra ausente é um caso em que o critério impediu um achado que a ferramenta não poderia sustentar. O argumento é o mesmo da literatura de análise estática, discutido no Capítulo 2: uma regra com precisão baixa é pior que uma regra ausente, porque compromete a confiança no relatório inteiro.

3.4 AS REGRAS

As 9 regras se dividem em três categorias: modelagem, desempenho e DAX. Cada uma é descrita abaixo pelo problema que aponta, pelo motivo de ele ser um problema, segundo a página do Microsoft Learn que a ancora, e pelo que a regra recomenda. Os conceitos de modelagem usados aqui, como relacionamento, cardinalidade, tabela de fatos e de dimensão, são apresentados no Capítulo 2. O catálogo completo, com o endereço de cada âncora, está no Apêndice A.

3.4.1 Regras de modelagem

As seis regras de modelagem leem a estrutura do modelo: as tabelas e os relacionamentos entre elas.

MOD-001 — tempo automático de data/hora ligado. Com essa opção ligada, o Power BI cria uma tabela de calendário oculta para cada coluna de data do modelo. Cada uma dessas tabelas aumenta o tamanho do modelo e o tempo de atualização dos dados, e, como a opção vale para todas as colunas de data ou para nenhuma, não se pode desligá-la só onde ela não serve (Microsoft, 2025d). A documentação recomenda mantê-la ligada apenas em modelos exploratórios e, nos demais, usar uma tabela de calendário própria. A regra aponta cada tabela de data gerada pelo produto.

MOD-002 — relacionamento com filtro bidirecional. Num relacionamento, o filtro normalmente se propaga num só sentido, da tabela de dimensão para a de fatos: filtrar um

produto restringe as vendas daquele produto. Quando o filtro é *bidirecional*, ele também volta no sentido contrário. A documentação recomenda minimizar esse uso, porque ele exige mais processamento, pode degradar o desempenho das consultas e torna o comportamento dos filtros confuso para quem usa o relatório (Microsoft, 2024e). Ela reconhece cenários legítimos, como a tabela-ponte de uma relação muitos-para-muitos, e mesmo nesses prefere ativar a propagação nos dois sentidos dentro de uma medida, com a função `CROSSFILTER`. A regra exclui os relacionamentos um-para-um, que são sempre bidirecionais e pertencem à MOD-007.

MOD-003 — tabela sem relacionamento. Uma tabela que não se relaciona com nenhuma outra não filtra nem é filtrada pelas demais, e um visual que combine os campos dela com os de outra tabela não produz o resultado esperado. A âncora é a orientação geral sobre esquema estrela, segundo a qual um modelo bem desenhado tem o número certo de tabelas com os relacionamentos devidos (Microsoft, 2024a). Por ser uma âncora geral, e não um artigo dedicado, esta é a regra de severidade mais baixa do conjunto. Ela recomenda verificar se a tabela precisa de relacionamento e, se ela só existe para apoiar outras consultas, desabilitar a sua carga no modelo.

MOD-005 — dimensão de data não marcada como tabela de data. Marcar uma tabela como tabela de data é dizer ao Power BI qual tabela é o calendário oficial do modelo. A marcação é obrigatória para as funções clássicas de inteligência temporal do DAX, como as de acumulado do ano, e, quando é feita, o Power BI remove as tabelas de data automáticas que havia criado (Microsoft, 2025e). A regra aponta a tabela de calendário própria do autor que ainda não foi marcada; a forma de provar a falta de marcação foi discutida na seção anterior.

MOD-006 — dimensão em floco de neve. No *floco de neve*, uma mesma entidade de negócio é guardada em várias tabelas encadeadas, como produto, subcategoria e categoria, cada uma ligada à seguinte. A documentação reconhece que essa pode ser uma escolha deliberada, mas diz que, em geral, os benefícios de integrar a entidade numa única tabela superam os de mantê-la em várias: o modelo carrega menos tabelas, as cadeias de propagação de filtro ficam mais curtas, quem constrói o relatório vê menos tabelas e passa a ser possível criar uma hierarquia que atravesse os níveis da entidade (Microsoft, 2024a). A regra aponta cada tabela intermediária de uma cadeia de dimensões, reconhecida por ser o lado "um" de um relacionamento e o lado "muitos" de outro.

MOD-007 — relacionamento um-para-um. Num relacionamento um-para-um, cada linha de uma tabela corresponde a no máximo uma linha da outra, o que costuma indicar que os dados de uma mesma entidade foram repartidos em duas tabelas. A documentação recomenda evitar esse desenho e consolidar as duas tabelas numa só, porque ele multiplica as tabelas que o autor do relatório precisa percorrer, dispersa campos relacionados, impede hierarquias entre os

níveis e produz resultados inesperados quando as linhas das duas tabelas não casam exatamente (Microsoft, 2024c). O cenário legítimo que ela reconhece, a dimensão degenerada derivada de uma tabela de fatos, consta da recomendação da regra.

3.4.2 Regras de desempenho

As duas regras de desempenho leem propriedades das colunas.

PERF-001 — coluna calculada em DAX. Uma coluna pode ser criada de duas formas: no Power Query, durante a carga dos dados, ou no próprio modelo, por uma fórmula DAX, a chamada *coluna calculada*. O resultado parece o mesmo, mas o VertiPaq, o mecanismo que armazena os dados do modelo comprimidos em memória, guarda as colunas calculadas em estruturas que normalmente comprimem menos, e só as constrói depois que todas as consultas do Power Query terminam de carregar, o que alonga a atualização (Microsoft, 2024b). A documentação recomenda, por isso, preferir criar a coluna no Power Query ou na origem dos dados, e reconhece exceções: a fórmula que avalia medidas ou exige funcionalidade que só existe em DAX, e as colunas que estendem uma tabela de data criada em DAX.

PERF-003 — coluna de ponto flutuante somada. O tipo *Número decimal* do Power BI guarda os valores em ponto flutuante, segundo o padrão IEEE 754, isto é, de forma aproximada, com até 15 dígitos de precisão. A documentação adverte que, raramente, somar uma coluna desse tipo pode devolver um resultado inesperado, sobretudo quando ela tem muitos valores positivos e negativos, porque o resultado passa a depender da distribuição dos valores entre as linhas, e que comparações de igualdade também podem surpreender (Microsoft, 2025b). A recomendação é trocar o tipo para *Número decimal fixo*, com quatro casas decimais exatas, ou *Número inteiro*. A regra aponta as colunas desse tipo cuja agregação padrão, a forma como o Power BI resume a coluna quando ela é arrastada para um visual, é a soma.

3.4.3 Regra de DAX

DAX-001 — divisão com o operador `/` e denominador não constante. Uma divisão por zero em DAX não interrompe o cálculo, mas produz um valor infinito ou indefinido que aparece no relatório. A função `DIVIDE` trata esse caso sozinha: quando o denominador é zero ou vazio, devolve vazio, ou um valor alternativo indicado pelo autor. A documentação recomenda usá-la sempre que o denominador for uma expressão que possa resultar em zero ou vazio, e lembra que ela é mais eficiente que testar o denominador antes com a função `IF` (Microsoft, 2021). A âncora mudou o enunciado da regra. A versão inicial, "divisão com `/` em vez de `DIVIDE`", marcaria como defeito o que a própria documentação recomenda quando o denominador é constante: usar o operador, que evita um teste desnecessário. A regra implementada afirma apenas sobre denominador não constante, condição que as duas passagens da página sustentam juntas.

A Tabela 2 resume o sinal que cada regra lê no `model.bim` e se ela afirma pela presença ou pela ausência de algo. Só a MOD-003 afirma por ausência, e pode fazê-lo porque o terceiro teste se cumpre por inteiro: todos os relacionamentos de um modelo estão no próprio arquivo, de modo que uma tabela ausente de todos eles está, de fato, desconectada. Mesmo assim, como mostra o Capítulo 4, a regra erra em tabelas desconectadas de propósito, um caso que o critério não previu.

Tabela 2 – O sinal que cada regra lê e a forma da afirmação

Regra	**Sinal lido no `model.bim`**	**Afirma por**
MOD-001	Tabela de data gerada pelo produto (tempo automático)	presença
MOD-002	Relacionamento com filtro cruzado nos dois sentidos, exceto um-para-um	presença
MOD-003	Tabela ausente de todos os relacionamentos do modelo	ausência
MOD-005	`variations` da coluna de data da dimensão apontando para uma tabela automática	presença
MOD-006	Tabela no lado “um” de um relacionamento e no lado “muitos” de outro	presença
MOD-007	Relacionamento com cardinalidade “um” nas duas pontas	presença
PERF-001	Coluna calculada em DAX, exceto colunas de agrupamento e colunas de uma dimensão de data calculada em DAX	presença
PERF-003	Coluna de ponto flutuante com agregação padrão de soma	presença
DAX-001	Operador `/` com denominador não constante, por expressão	presença

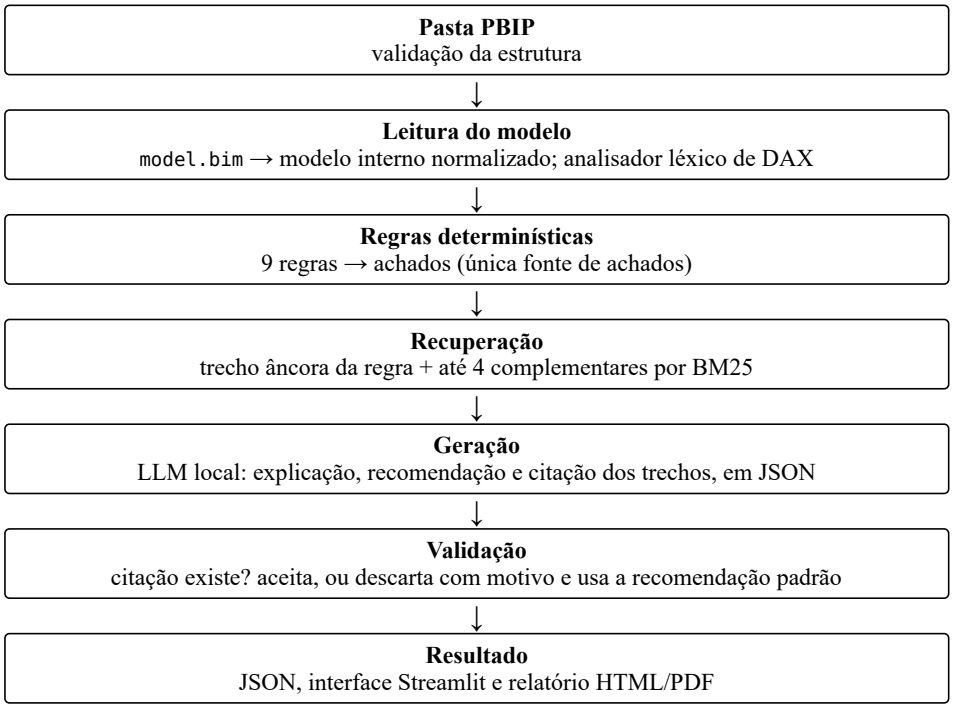
Fonte: Elaborado pelo autor a partir do código das regras (`core/rules/`).

As exceções da PERF-001 também vêm da documentação: as colunas de agrupamento têm o DAX escrito pela própria interface, e as páginas sobre tempo automático e tabelas de data recomendam criar a tabela de data com `CALENDAR` ou `CALENDARAUTO` e acrescentar colunas calculadas a ela (Microsoft, 2025d). Sem essas exceções, a regra recomendaria desfazer o que a fonte recomenda fazer.

3.5 FLUXO DA AUDITORIA

A auditoria é uma função que executa as etapas em sequência, sem orquestração de agentes (ADR-003), conforme a Figura 2. O fluxo é linear e um arcabouço de agentes não traria ganho demonstrável; a sequência fixa também facilita explicar e testar cada etapa isoladamente.

Figura 2 – Fluxo da auditoria



Fonte: Elaborado pelo autor.

As regras são executadas em ordem estável e isoladas umas das outras: se uma regra falha com erro, ela não interrompe a auditoria, e sua saída parcial é descartada inteira, para que nenhum achado venha de uma execução incompleta. Antes de chamar o modelo de linguagem, a auditoria verifica que toda regra tem gabarito de âncora e que a seção âncora existe no índice; faltando qualquer uma, ela para em vez de gerar uma explicação sem fundamento. Além da linha de comando, a ferramenta tem uma interface em Streamlit e gera um relatório em HTML e PDF.

A Tabela 3 lista as tecnologias usadas (ADR-002). A escolha seguiu duas restrições do ambiente: tudo precisava instalar no Windows 11 sem compilador C++, e o modelo de linguagem precisava caber numa GPU de 6 GB. A recuperação foi escrita sem arcabouço, como um fluxo curto e próprio, porque um arcabouço como LangChain ou LlamaIndex acrescentaria abstrações que um fluxo linear não pede e que dificultariam explicar o funcionamento.

Tabela 3 – Pilha tecnológica da ferramenta

Camada	**Escolha**
Linguagem	Python 3.11, com Pydantic para o modelo interno validado
Leitura do modelo	`json` da biblioteca padrão sobre o `model.bim`; analisador léxico de DAX próprio
Recuperação	Fluxo próprio: coleta em HTML, extração por seção, índice BM25 (`rank-bm25`)
Modelo de linguagem	`qwen2.5:7b-instruct` quantizado em 4 bits, executado pelo Ollama na GPU local
Interface	Streamlit
Relatório	Jinja2 para o HTML; Playwright controlando o Microsoft Edge do sistema para o PDF
Testes e avaliação	pytest; pacotes próprios para o gabarito, a medição da recuperação e a rubrica da geração

Fonte: Elaborado pelo autor a partir da ADR-002.

Todo o desenvolvimento foi guiado por testes automatizados, escritos antes do código. Além dos testes de unidade, há testes que conferem artefatos contra a sua fonte: o catálogo de fontes da recuperação é regenerado e comparado com o arquivo versionado, a passagem citada por cada regra é procurada no texto extraído da base, e as tabelas de resultado desta monografia são geradas dos arquivos de resultado e comparadas com as que o texto inclui. Assim, um número que mude na origem faz a suíte falhar até que o documento seja atualizado.

3.6 BASE DE DOCUMENTAÇÃO E RECUPERAÇÃO

A base consultada pela recuperação é a documentação pública do Microsoft Learn. O plano inicial de clonar o repositório de origem da documentação não se sustentou, porque o repositório é privado. As páginas foram então coletadas em HTML a partir dos índices publicados de cada seção do site, depois de verificado que o arquivo `robots.txt` não bloqueia o caminho (ADR-004). O catálogo de fontes é gerado desses índices, e não digitado, e reúne 1.420 páginas: a seção de orientação do Power BI inteira, as referências completas de funções DAX e de Power Query M e as páginas âncora das regras. A cópia local das páginas fica fora do repositório; versionam-se o catálogo e o código de coleta. Seis artigos do SQLBI entram apenas como leitura complementar, com link, porque os termos do site não autorizam armazenar os artigos.

Cada página é extraída por seção, preservando o endereço da seção dentro da página e o caminho de títulos até ela. As seções são os *trechos* que a busca devolve, com até 350 palavras; a divisão do texto em trechos, chamada *fragmentação*, define a unidade que o modelo de linguagem recebe e cita. A recuperação tem dois níveis (ADR-002, emenda de 09/10/2026). O primeiro é fixo: a **seção âncora** da regra, a passagem transcrita quando a regra passou no segundo teste do critério, entra sempre como o trecho número 1. O segundo vem da busca, que acrescenta trechos complementares. Essa construção nasceu de uma falha observada no teste inicial do modelo de linguagem: com várias páginas no contexto, o modelo citou uma página válida, mas menos pertinente que a âncora. Com a âncora fixa, a passagem que sustenta o achado está sempre disponível, e a qualidade da busca passa a afetar só os complementares.

O método de busca foi escolhido por medição, com a régua definida antes de medir. Três tipos de busca foram considerados. A busca *lexical* BM25 (Robertson; Zaragoza, 2009) ordena os trechos pelas palavras que têm em comum com a consulta, dando mais peso às palavras raras no corpus. A busca *densa* converte consulta e trechos em vetores numéricos que representam o significado do texto, com o modelo `bge-small-en-v1.5`, e ordena pela proximidade entre os vetores, o que permite achar um trecho que diga a mesma coisa com outras palavras. A busca *híbrida* combina as duas ordenações pela fusão recíproca de posições (Cormack; Clarke;

Büttcher, 2009), que soma, para cada trecho, o inverso da sua posição em cada lista. Foram comparadas 24 combinações de fragmentação (por seção ou por janela de palavras), cabeçalho de contexto (com ou sem os títulos da seção repetidos no início do trecho), tipo de busca e corpus (completo ou só orientação). A regra de decisão, fixada antes, escolhia a maior taxa de acerto entre os cinco primeiros resultados no corpus completo e, em caso de empate próximo, a combinação mais simples. Venceu a fragmentação por seção, sem cabeçalho, com BM25 sobre o corpus completo. Os números e as limitações dessa medição estão no Capítulo 4.

3.7 GERAÇÃO COM CITAÇÃO OBRIGATÓRIA

A explicação é redigida por um *modelo de linguagem* (*large language model*, LLM), um modelo que gera texto prevendo, a cada passo, a continuação mais provável do que recebeu. O modelo usado é o `qwen2.5:7b-instruct` (Qwen Team, 2024), com 7 bilhões de parâmetros, executado localmente pelo Ollama, um programa que serve modelos abertos na própria máquina, numa GPU de 6 GB. Para caber na memória da GPU, o modelo é *quantizado* em 4 bits: seus parâmetros são guardados com menos precisão numérica, o que reduz a memória necessária a cerca de um quarto, com alguma perda de qualidade. O modelo foi escolhido num teste comparativo com a versão de 3 bilhões de parâmetros: os dois citaram corretamente nos cinco casos de teste, mas o menor errou um número que estava na própria evidência. A execução local tem uma razão prática, além do custo: o modelo semântico de um cliente não precisa sair da máquina de quem o audita. O modelo lê e escreve em *tokens*, pedaços de palavra que servem de unidade ao modelo, e a *janela de contexto* é o número máximo de tokens que ele considera de uma vez, somando o pedido e a resposta. Ela foi fixada em 4.096 tokens, o máximo que cabe na memória da GPU sem descarregar parte do modelo para a CPU, o que o tornaria muito mais lento. A geração usa *temperatura* 0, o parâmetro que controla o quanto o modelo se afasta da continuação mais provável, e uma semente fixa para o sorteio, ambas escolhidas para reduzir a variação entre execuções.

O pedido ao modelo tem duas partes. A instrução diz que o problema já foi detectado por uma regra determinística e que o modelo não decide se ele existe; que deve usar só os trechos fornecidos; citar de 1 a 3 trechos pelo número; e responder em português do Brasil, de forma curta e acionável. A mensagem traz a regra, o objeto afetado, a evidência do achado e os trechos numerados, em inglês, a língua do corpus. A resposta tem de seguir um *esquema JSON*, uma descrição formal dos campos esperados e de seus tipos, que o Ollama impõe durante a geração: explicação e recomendação como texto e as citações como uma lista de 1 a 3 números inteiros. Assim, a resposta sempre pode ser lida por código. A Figura 3 reproduz a instrução de sistema, igual em todos os achados. A citação é pedida pelo número do trecho, e não pelo endereço da

página, porque um modelo de 7 bilhões de parâmetros copia mal cadeias longas; o código traduz o número para página e seção.

Figura 3 – Instrução de sistema enviada ao modelo de linguagem

> Você é um auditor de modelos semânticos do Power BI.
> Uma regra determinística JÁ detectou o problema descrito pelo usuário. Você NÃO decide se o problema existe: sua tarefa é explicá-lo e recomendar a correção.
> Use somente as informações do achado e dos trechos de documentação numerados. Não invente números, nomes de objetos nem fatos.
> Responda em português do Brasil, em JSON, com:
> - "explicacao": por que isso é um problema neste modelo;
> - "recomendacao": o que fazer, de forma acionável;
> - "citacoes": de 1 a 3 números dos trechos que sustentam a explicação.
> Explicação e recomendação juntas devem ter no máximo 120 palavras.

Fonte: Código da ferramenta (`core/analise/prompt.py`).

A validação é feita por código. Se a resposta não é JSON válido, se um campo vem vazio, se o número de citações sai do intervalo ou se algum número citado não existe no contexto, a resposta é descartada, sem nova tentativa, e o motivo é registrado. O achado continua no resultado com a recomendação padrão da regra e a marca "sem fundamentação gerada": o descarte nunca apaga um achado. Uma resposta aceita passa ainda por uma conferência de superfície, que verifica se números e nomes de objetos citados na explicação aparecem no achado ou nos trechos. Por ser uma heurística ainda não calibrada, ela registra divergências, mas não descarta. O Capítulo 4 mostra que essa conferência não mede o que a leitura humana chama de fidelidade.

As respostas podem ser guardadas num cache, indexado pelo pedido completo, pelo modelo e pelos parâmetros de geração. O cache torna a auditoria repetível em segundos com textos idênticos, o que é necessário porque a temperatura 0 não torna a geração do Ollama determinística: entre duas execuções, os totais se repetem, mas os textos e algumas citações variam. Ele vem desligado por padrão e, quando usado, é indicado no resultado e no relatório.

3.8 CONJUNTO DE AVALIAÇÃO

As métricas vêm de sete projetos públicos do repositório de exemplos do Power BI Desktop mantido pela Microsoft, sob licença MIT, convertidos manualmente para PBIP em TMSL (ADR-005). Eles cobrem sete domínios: vendas, despesas corporativas, recursos humanos, marketing, varejo, cadeia de suprimentos e funil comercial. Cinco vêm da revisão de 2026 dos exemplos e dois de amostras mais antigas, uma mistura que evita um conjunto uniformemente bom ou ruim. Por serem públicos, qualquer leitor pode refazer a avaliação com os mesmos arquivos. Esses projetos são chamados de P1 a P7.

Um oitavo projeto, P8, é um modelo de autoria própria sobre vendas de varejo online. Ele chegou a integrar o conjunto de métricas, mas saiu dele porque não pode ser reproduzido por terceiros, e passou a estudo de caso: mostra a ferramenta agindo em profundidade sobre um

modelo com problemas reais, a começar pelo tempo automático ligado ao lado de uma dimensão de data própria. Nenhuma métrica do Capítulo 4 depende do P8.

O gabarito das regras foi anotado pelo autor. Para cada regra, um gerador independente da lógica da regra lista todos os objetos que poderiam violá-la — por exemplo, todas as tabelas, para a MOD-003, ou todas as colunas calculadas, para a PERF-001 —, acompanhados de fatos neutros sobre cada um. A anotação foi cega: a planilha não informava quais objetos a ferramenta marcara, e o anotador julgava cada um contra a pergunta da âncora, com os vereditos “viola”, “não viola” ou “indeterminado”, este último reservado aos casos em que o modelo não traz a informação necessária. Nenhuma regra mudou durante a anotação, e a versão medida foi congelada numa marca de versão do repositório no commit que a encerrou. O protocolo previa criar a marca antes de anotar; ela foi criada depois, mas sobre o mesmo código. Correções de regra feitas a partir daí são relatadas como análise de erros, medidas contra o mesmo gabarito, e não como resultado.

4 AVALIAÇÃO

A avaliação responde a três perguntas, uma para cada camada da ferramenta. As regras apontam os problemas certos? A recuperação encontra a passagem da documentação que sustenta cada achado? A explicação gerada pode ser entregue ao dono do modelo sem edição? Este capítulo apresenta o desenho da avaliação, os resultados de cada pergunta e o estudo de caso do projeto P8. Todas as tabelas de resultado foram geradas pelo código a partir dos arquivos de resultado versionados no repositório, que o leitor pode conferir.

4.1 DESENHO DA AVALIAÇÃO

Cada pergunta tem a sua medida, o seu gabarito e uma regra de decisão escrita antes da medição. Fixar a regra antes de medir impede que a escolha se ajuste ao resultado: quando o resultado contrariou a expectativa, a regra foi mantida e a divergência, declarada. Esta seção define as medidas usadas no restante do capítulo.

4.1.1 Achado, gabarito e os quatro desfechos

Um **achado** é a afirmação da ferramenta de que um objeto do modelo — uma tabela, uma coluna, um relacionamento ou uma expressão DAX — viola uma regra. O **gabarito** é o julgamento humano, feito de forma independente, sobre cada objeto que poderia violar aquela regra, quer a ferramenta o tenha marcado, quer não. Cruzar os dois produz quatro desfechos possíveis para cada objeto:

- **verdadeiro positivo** (VP): a ferramenta marcou o objeto e o gabarito confirma a violação;
- **falso positivo** (FP): a ferramenta marcou o objeto, mas o gabarito diz que ele não viola a regra;
- **falso negativo** (FN): a ferramenta não marcou o objeto, mas o gabarito diz que ele viola a regra;
- **verdadeiro negativo** (VN): a ferramenta não marcou e o gabarito confirma que não há violação.

Um exemplo do próprio conjunto de avaliação ajuda a fixar os termos. No projeto P6, a tabela `Logo` não tem nenhum relacionamento, e a MOD-003, que aponta tabelas sem relacionamento, a marcou. O anotador, porém, julgou que ela não viola a regra: é uma tabela criada à mão só para exibir o logotipo no relatório, e não precisa se relacionar com nada. É, portanto, um falso positivo.

4.1.2 Precisão e revocação

A **precisão** é a fração dos achados emitidos que estão certos, VP / (VP + FP). Ela responde à pergunta de quem recebe o relatório: quando a ferramenta aponta um problema, posso confiar? A **revocação** é a fração dos problemas existentes que a ferramenta encontrou,

VP / (VP + FN). Ela responde à pergunta de quem quer cobertura: quantos problemas ficaram de fora? As duas medidas costumam competir, porque uma regra mais permissiva encontra mais problemas e também erra mais. Pelo argumento do Capítulo 3, este trabalho dá prioridade à precisão: um relatório com muitos alarmes falsos perde a confiança de quem o lê.

Quando há várias regras, as medidas podem ser agregadas de duas formas. A média **micro** soma os desfechos de todas as regras antes de calcular a proporção, de modo que uma regra com muitos achados pesa mais. A média **macro** calcula a proporção de cada regra e depois tira a média entre elas, de modo que cada regra pesa igual. A primeira descreve a experiência de quem lê o relatório inteiro; a segunda, a qualidade típica de uma regra.

4.1.3 Casos indeterminados: principal, piso e teto

Alguns objetos não podem ser julgados a partir do que o modelo contém. Na MOD-005, por exemplo, o `model.bim` não registra se uma tabela foi marcada como tabela de data, de modo que, com o tempo automático desligado, não há como afirmar se a marcação foi feita. Nesses casos o anotador registra **indeterminado**. A proporção **principal** é calculada só sobre os casos decididos. Ela vem acompanhada de dois limites: o **piso**, que conta todos os indeterminados contra a ferramenta, e o **teto**, que os conta a favor. Se o piso e o teto ficam próximos da principal, os indeterminados não mudam a conclusão.

4.1.4 Intervalo de confiança de Wilson

Uma proporção medida sobre poucos casos é incerta: acertar o único achado de uma regra não prova que a regra acerta sempre. O **intervalo de confiança de 95%** expressa essa incerteza como uma faixa de valores compatíveis com o que foi observado. Foi usado o intervalo de Wilson (Wilson, 1927), que se comporta bem justamente nas situações frequentes aqui, amostras pequenas e proporções perto de 0 ou de 1, nas quais o intervalo mais conhecido, baseado na aproximação normal, produz faixas que ultrapassam os limites de 0 e 1 ou se reduzem a um ponto. Por exemplo, um acerto em um caso dá precisão 1,00, mas com intervalo de 0,21 a 1,00: o dado é compatível tanto com uma regra excelente quanto com uma regra que erra quatro vezes em cinco.

4.1.5 Concordância e kappa de Cohen

Para saber se um critério automático substitui o julgamento humano, compara-se os dois caso a caso. A **concordância simples** é a fração de casos em que os dois dizem a mesma coisa. Ela tem um defeito: se quase todos os casos são “sim”, dois julgadores que respondem “sim” sempre concordam muito sem que um saiba nada do outro. O **kappa de Cohen** (Cohen, 1960) corrige isso, descontando a concordância que se esperaria ao acaso, dadas as proporções de

"sim" e "não" de cada julgador. Vale 1 quando a concordância é perfeita, 0 quando ela é a que o acaso produziria, e fica negativo quando os dois concordam menos do que o acaso.

4.2 REGRAS

O gabarito das regras tem 350 linhas, uma por objeto candidato a violar uma regra nos projetos P1 a P7. Um objeto é **candidato** quando é do tipo que a regra examina: para a MOD-003, toda tabela; para a PERF-001, toda coluna calculada; para a DAX-001, toda expressão que contém o operador de divisão. A anotação foi cega, como descrito no Capítulo 3: o anotador julgou cada candidato sem saber se a ferramenta o marcara. A primeira medição levou a uma releitura das divergências, e 12 julgamentos foram revistos com base na documentação, cada um com o motivo registrado e o julgamento original preservado no histórico do repositório. A revisão andou nos dois sentidos. Sete linhas da MOD-005 passaram de "viola" a "indeterminado", porque o `model.bim` não registra a marcação de tabela de data e não havia como afirmar que a tabela não estava marcada. Três linhas da DAX-001 passaram de "não viola" a "viola", porque a página da âncora usa exatamente aquele padrão como exemplo do que deve ser trocado pela função `DIVIDE` (Microsoft, 2021). O padrão é a divisão **protegida** por um teste, como em `IF(denominador <> 0, numerador / denominador, BLANK())`: a expressão só divide quando o denominador é diferente de zero e, caso contrário, devolve vazio. O anotador tinha julgado que a proteção afastava o problema; a documentação diz que `DIVIDE` faz o mesmo teste de forma mais eficiente que a função `IF`. As demais revisões foram uma coluna gerada pelo recurso de agrupamento do Power BI e uma tabela de negócio sem nenhum relacionamento. A Tabela 4 mostra o resultado oficial, com as regras na versão congelada.

Tabela 4 – Desempenho das regras contra o gabarito em P1–P7 (resultado oficial (tag regras-gt))

Regra	VP	FP	FN	Indet.	Precisão	IC 95% (Wilson)	Revocação
DAX-001	6	0	0	0	1,00	[0,61; 1,00]	1,00
MOD-001	1	0	0	1	1,00	[0,21; 1,00]	1,00
MOD-002	0	0	0	0	—	—	—
MOD-003	1	6	0	0	0,14	[0,03; 0,51]	1,00
MOD-005	0	0	0	7	—	—	—
MOD-006	3	0	0	0	1,00	[0,44; 1,00]	1,00
MOD-007	1	0	0	0	1,00	[0,21; 1,00]	1,00
PERF-001	18	0	6	5	1,00	[0,82; 1,00]	0,75
PERF-003	16	0	0	0	1,00	[0,81; 1,00]	1,00
Total (micro)	46	6	6	13	0,88	[0,77; 0,95]	0,88

Fonte: Elaborado pelo autor a partir de `regras-2026-10-12.json`.

A precisão total foi de 0,88: dos 52 achados emitidos pelas regras, 46 foram confirmados pelo gabarito. A revocação total também foi de 0,88. Seis das nove regras tiveram precisão 1,00. Duas não emitiram achado algum: a MOD-002 porque nenhum dos sete projetos tem relacionamento bidirecional que ela cubra, e o gabarito confirmou a ausência, e a MOD-005 porque,

com o tempo automático desligado, ela se cala por desenho, o falso negativo que o Capítulo 3 declarou; os sete casos que o anotador não pôde decidir aparecem na coluna de indeterminados. A amplitude dos intervalos de Wilson mostra o quanto o número de casos limita as conclusões por regra: a MOD-001 e a MOD-007 têm precisão 1,00 sobre um único achado, com intervalo de 0,21 a 1,00. As regras com mais casos, PERF-001 e PERF-003, têm intervalos estreitos, com limite inferior acima de 0,80.

O projeto fixou, entre os seus riscos, um limiar de precisão de 0,7 por regra, abaixo do qual a regra seria revista. A referência industrial é mais exigente: as ferramentas de análise estática do Google só mantêm uma verificação se menos de 10% dos seus avisos forem **falsos positivos efetivos** (Winters; Manshreck; Wright, 2020). O conceito é definido pelo efeito sobre quem recebe o aviso, e não pela correção técnica: um aviso é falso positivo efetivo quando o desenvolvedor não toma nenhuma providência depois de vê-lo, seja porque o aviso está errado, seja porque ele não o entendeu. Por esse critério, uma explicação confusa transforma um achado correto em falso positivo efetivo, o que liga a qualidade da geração, avaliada adiante, à utilidade das regras. Só uma regra ficou abaixo do limiar: a MOD-003, com precisão de 0,14, um acerto e seis falsos positivos.

A análise desses seis falsos positivos mostrou duas causas. Quatro eram objetos que o Power BI cria desconectados de propósito. Dois eram **grupos de cálculo**, tabelas especiais cujas linhas são fórmulas DAX aplicadas sobre as medidas do modelo, por exemplo para mostrar qualquer medida no acumulado do ano sem escrever uma medida nova para cada caso; elas agem sobre as medidas, e não sobre as linhas de outra tabela, por isso não têm relacionamento. Os outros dois eram **parâmetros de campo**, tabelas que o Power BI gera para que o leitor do relatório escolha, num seletor, qual medida ou coluna um visual exibe; também elas funcionam sem relacionamento. Os dois tipos têm sinal estrutural próprio no `model.bim`: a propriedade `calculationGroup`, no primeiro caso, e uma propriedade de metadados de parâmetro, no segundo. Os outros dois, `Logo` e `Indicators`, eram tabelas digitadas à mão para a apresentação do relatório, sem nenhum sinal que as distinga de uma dimensão digitada e esquecida. Na PERF-001, os seis falsos negativos tinham uma causa só: a exceção para colunas de tabelas de data estava larga demais e excluía também as tabelas de data importadas pelo Power Query, quando a documentação só recomenda colunas calculadas na tabela de data criada em DAX.

As duas regras foram corrigidas depois da medição oficial, sempre por sinal estrutural e nunca pelo nome do objeto, e medidas de novo contra o mesmo gabarito (Tabela 5). Esse resultado é **análise de erros**, não resultado: as correções foram motivadas pelos próprios casos em que são medidas, e a sua validação exigiria projetos que as regras ainda não viram.

Tabela 5 – Desempenho das regras após a correção da MOD-003 e da PERF-001 (análise de erros)

Regra	VP	FP	FN	Indet.	Precisão	IC 95% (Wilson)	Revocação
DAX-001	6	0	0	0	1,00	[0,61; 1,00]	1,00
MOD-001	1	0	0	1	1,00	[0,21; 1,00]	1,00
MOD-002	0	0	0	0	—	—	—
MOD-003	1	2	0	0	0,33	[0,06; 0,79]	1,00
MOD-005	0	0	0	7	—	—	—
MOD-006	3	0	0	0	1,00	[0,44; 1,00]	1,00
MOD-007	1	0	0	0	1,00	[0,21; 1,00]	1,00
PERF-001	24	2	0	5	0,92	[0,76; 0,98]	1,00
PERF-003	16	0	0	0	1,00	[0,81; 1,00]	1,00
Total (micro)	52	4	0	13	0,93	[0,83; 0,97]	1,00

Fonte: Elaborado pelo autor a partir de `regras-2026-10-12-pos-correcao.json`.

A MOD-003 subiu para 0,33 e continua abaixo do limiar, por causa das duas tabelas digitadas, uma limitação declarada da regra. A PERF-001 passou a ter revocação 1,00 e precisão 0,92. Parte desse ganho, porém, está em casos indeterminados que a regra passou a marcar: contados contra ela, a precisão da PERF-001 cai para 0,77.

O conjunto de avaliação também precisava de achados suficientes. O planejamento fixou um mínimo de cerca de 40 achados confirmados em P1 a P7, abaixo do qual um oitavo projeto seria acrescentado. Os 46 achados confirmados superam o mínimo, mas 16 deles vêm de uma única regra, a PERF-003; sem ela, seriam 30.

4.3 RECUPERAÇÃO

A recuperação foi avaliada em duas etapas. A primeira escolheu o método de busca; a segunda, feita depois que a âncora passou a entrar sempre no contexto, verificou se a busca entregava trechos complementares úteis.

Na primeira etapa, o gabarito de cada regra é a sua **seção âncora**: a seção da página do Microsoft Learn que contém a passagem transcrita quando a regra foi aprovada. Para reconhecer a seção âncora entre os resultados sem depender de julgamento, cada gabarito guarda uma **frase-marca**, um trecho literal da passagem; um resultado acerta quando contém essa frase. A **consulta** é o texto que a ferramenta envia à busca: os termos de pesquisa da regra, em inglês, acrescidos dos dados do achado, como o nome do objeto. Foram 62 consultas, uma por regra e uma por achado de P1 a P7.

A busca devolve uma lista ordenada de seções, da mais à menos relevante segundo ela. Três medidas descrevem essa lista. O **acerto@K** vale 1 quando a seção âncora aparece entre as K primeiras posições e 0 caso contrário; com $K = 5$, por exemplo, ele pergunta se a âncora estaria entre os cinco trechos que caberiam no contexto do modelo. A **precisão@K** é a fração das K primeiras seções que são a âncora; como cada consulta tem uma única seção âncora, ela vale no máximo $1/K$, e por isso cai à medida que K cresce. O **MRR** (*mean reciprocal rank*)

é a média do inverso da posição em que a âncora aparece: vale 1 quando ela está em primeiro lugar, 0,5 quando está em segundo, 0,33 em terceiro, e assim por diante, até a décima posição, a partir da qual conta zero. Ele premia a busca que põe a âncora mais acima. As médias são **macro**: calculadas primeiro entre as consultas de cada regra e depois entre as nove regras, para que a PERF-001, com mais achados, não domine o resultado. A Tabela 6 mostra a combinação vencedora entre as 24 comparadas.

Tabela 6 – Recuperação vencedora (fragmento por seção, busca BM25, corpus completo): médias macro por regra em 62 consultas

K	Acerto@K	Precisão@K
1	0,44	0,44
3	0,72	0,24
5	1,00	0,20
10	1,00	0,10
MRR@10	0,61	—

Fonte: Elaborado pelo autor a partir de `recuperacao-2026-10-11.json`.

A seção âncora apareceu entre os cinco primeiros resultados em todas as regras, e na primeira posição em menos da metade delas. O número precisa de duas ressalvas, registradas na decisão. A primeira é um **efeito teto**, nome dado à situação em que muitas alternativas atingem o valor máximo da medida e ela deixa de distinguir a melhor: 7 das 12 combinações do corpus completo chegaram ao acerto@5 de 1,00. Isso acontece porque as consultas carregam os termos da própria regra, que também aparecem na passagem âncora, o que torna a tarefa mais fácil do que uma pergunta livre de um usuário seria. A segunda é que uma única consulta decidiu o empate: a busca densa ficou fora dele só porque a consulta da MOD-005 encontrou a âncora na sétima posição, e essa regra, com uma consulta, pesa um nono da média. A ampliação do corpus para as referências completas de DAX e M, por outro lado, não prejudicou a recuperação: o corpus restrito à orientação não superou o completo.

A segunda etapa mudou a pergunta. Com a âncora fixa como primeiro trecho, a busca só escolhe os **trechos complementares**, do segundo ao quinto, que acrescentam contexto à âncora. A medida anterior, que procurava a âncora, deixou de informar sobre eles. A nova medida é a **pertinência**: a fração dos complementares que de fato ajudam a explicar o achado, segundo o julgamento do autor. Foram comparadas as três buscas descritas no Capítulo 3: a lexical BM25, a densa e a híbrida. Para não julgar a mesma seção várias vezes nem favorecer uma busca, as seções entregues pelas três foram reunidas num só conjunto, técnica chamada de *pooling*: o autor julgou as 89 seções distintas entregues em 70 consultas, sem saber qual busca tinha entregado cada uma. A regra, fixada antes, só trocaria o BM25 por outra busca se ela superasse a média do BM25 em pelo menos 0,10 (Tabela 7).

Tabela 7 – Pertinência dos trechos complementares por estratégia de busca (média macro)

Estratégia	Principal	Piso	Teto
BM25	0,62	0,59	0,62
Densa (bge-small)	0,67	0,64	0,71
Híbrida	0,65	0,64	0,69

Fonte: Elaborado pelo autor a partir de `complementares-2026-10-12.json`.

Nenhuma alternativa superou o BM25 pela margem fixada, e ele foi mantido. A medição mostrou um problema que nenhuma troca de busca resolveria: na PERF-001, nenhuma das três buscas entregou complemento pertinente. A consulta dessa regra leva o nome da coluna, e 13 das 31 colunas estão em tabelas de data, com nomes como `Year`, `Month` ou `Mth of year`. A explicação provável é que esses nomes coincidem com nomes e termos de funções DAX: as buscas trouxeram páginas da referência de funções, como a de `YEARFRAC`, sem relação com o achado. O defeito, portanto, está na construção da consulta, não no tipo de busca. Registrou-se também que, considerando só as consultas por achado, que são as que a ferramenta de fato faz, a busca densa superaria o BM25 por 0,13; a regra aprovada antes da medição usava as 70 consultas e foi mantida.

4.4 GERAÇÃO

A geração foi avaliada sobre todos os 61 achados que as regras emitem em P1 a P7, sem amostragem. Como a geração depende só de quais achados existem, e não de como as regras os detectaram, foi usada a versão corrigida das regras; o resultado oficial da detecção continua sendo o da Tabela 4. Os 61 achados receberam explicação: nenhuma resposta foi descartada pela validação da citação. A rodada levou 8 minutos e 53 segundos e foi gravada no cache, de modo que repeti-la reproduz exatamente os textos avaliados.

Cada explicação foi julgada pelo autor numa planilha que não mostrava nenhum dos sinais automáticos, para que a comparação entre os dois fosse válida. Três itens foram julgados com “sim”, “não” ou “indeterminado”, com justificativa obrigatória no “não”:

- **pertinente**: pelo menos um trecho citado é a passagem da documentação que sustenta o achado, e não apenas um trecho existente no contexto;
- **fiel**: tudo o que a explicação afirma é sustentado pelos trechos citados ou pelos fatos do achado, sem números, nomes ou comportamentos inventados;
- **recomendação correta**: a recomendação resolve o problema neste modelo, sem erro técnico.

Uma explicação é **utilizável como está** quando recebe “sim” nos três itens. Não houve casos indeterminados. A Tabela 8 mostra o resultado.

Tabela 8 – Rubrica humana das explicações geradas: proporção de “sim” por item

Item	**Sim / n**	**Proporção**	**IC 95% (Wilson)**
Pertinente	61 / 61	1,00	[0,94; 1,00]
Fiel	32 / 61	0,52	[0,40; 0,64]
Recomendação correta	47 / 61	0,77	[0,65; 0,86]
Utilizável como está	23 / 61	0,38	[0,27; 0,50]

Fonte: Elaborado pelo autor a partir de `geracao-2026-10-14.json`.

A recuperação e a citação funcionaram: todas as 61 explicações citaram uma passagem pertinente. O problema está no que o modelo escreve a partir dela. Pouco mais da metade das explicações foi fiel aos trechos, e pouco mais de um terço pode ser entregue sem edição. A Tabela 9 mostra que os erros se concentram em duas regras.

Tabela 9 – Rubrica humana das explicações por regra

Regra	**Achados**	**Fiel**	**Recomendação correta**	**Utilizável**
DAX-001	6	1 / 6	6 / 6	1 / 6
MOD-001	1	0 / 1	0 / 1	0 / 1
MOD-003	3	3 / 3	0 / 3	0 / 3
MOD-006	3	2 / 3	2 / 3	1 / 3
MOD-007	1	1 / 1	0 / 1	0 / 1
PERF-001	31	9 / 31	24 / 31	6 / 31
PERF-003	16	16 / 16	15 / 16	15 / 16

Fonte: Elaborado pelo autor a partir de `geracao-2026-10-14.json`.

Os “não” seguem padrões, e não se distribuem ao acaso. Na PERF-001, que responde por metade dos achados, a maior parte das explicações infiéis repete a mesma distorção: o trecho diz que o VertiPaq **comprime** com menos eficiência as colunas calculadas, e a explicação diz que ele não as “compreende” eficientemente, ou fala em “descompressão”. Tudo indica um erro de tradução do modelo, que escreve em português sobre uma fonte em inglês. As recomendações erradas da PERF-001 propõem sintaxe DAX para o Power Query ou funções de M que não existem. Na DAX-001, cinco das seis explicações tratam como defeito o fato de `DIVIDE` devolver vazio quando o denominador é zero. O vazio, `BLANK` no DAX, é o valor que representa a ausência de dado, e um visual simplesmente não exibe a linha de uma medida que devolve vazio; o trecho citado diz que esse é, em geral, o comportamento desejável para uma medida. Na MOD-003, as três recomendações mandam relacionar tabelas auxiliares que são desconectadas de propósito, o mesmo ponto cego da regra. Em contraste, a PERF-003 teve 15 de 16 explicações utilizáveis.

Por fim, a Tabela 10 compara o julgamento humano com os dois critérios automáticos da ferramenta: se a explicação cita a seção âncora e se a conferência de números e nomes não acusou divergência.

Tabela 10 – Concordância entre a leitura humana e a conferência automática (a: ambos sim; b: só o automático diz sim; c: só o humano diz sim; d: ambos não)

Item humano	Critério automático	a	b	c	d	Concordância	Kappa
Pertinente	Cita a seção âncora	60	0	1	0	0,98	0,00
Fiel	Sem aviso de números ou nomes	24	24	8	5	0,48	−0,08

Fonte: Elaborado pelo autor a partir de `geracao-2026-10-14.json`.

No item pertinente, a concordância foi de 0,98, mas o kappa foi zero. Não há contradição: quase todos os casos caíram na mesma célula, ambos "sim", e o kappa, que desconta a concordância esperada ao acaso, perde o sentido quando uma categoria domina a tabela, um paradoxo conhecido da medida (Feinstein; Cicchetti, 1990). O que se pode afirmar é mais simples: a combinação da âncora fixa com a citação obrigatória entregou evidência pertinente em todos os achados, e o único caso em que o modelo não citou a âncora citou outra passagem que também sustentava o achado.

No item fiel, a conferência automática concordou com o humano em menos da metade dos casos, com kappa de −0,08, nenhum acordo além do acaso. Em 24 explicações a conferência não viu problema e a leitura humana encontrou infidelidade. A razão é que as infidelidades reais são de sentido, como "compreende" no lugar de "comprime", e não números ou nomes trocados, que é o que a conferência verifica. O resultado mudou a ferramenta: o aviso, que se chamava "divergência de fidelidade", passou a se chamar "números ou nomes sem origem nos trechos", e toda explicação gerada passou a vir com uma nota dizendo que a ausência de aviso não garante a correção do texto.

4.5 ESTUDO DE CASO: O PROJETO P8

O P8 é um modelo de vendas de varejo online, de autoria própria, com 19 tabelas, 93 medidas e 35 colunas calculadas. Ele foi o primeiro projeto auditado e o que motivou várias decisões de método, mas não entra nas métricas, porque não pode ser reproduzido por terceiros. A ferramenta encontra nele 15 achados de oito regras. O mais característico é o tempo automático ligado. O **tempo automático de data/hora** é uma opção do Power BI Desktop que, para cada coluna de data do modelo, cria uma tabela oculta de calendário, com ano, trimestre, mês e dia, para que o autor do relatório possa agrupar e navegar pelas datas sem construir um calendário. O Power BI cria uma tabela-modelo (`DateTableTemplate`) e, a partir dela, uma tabela local (`LocalDateTable`) para cada coluna de data. O P8 tem uma dimensão de data própria, `DimCalendar`, uma tabela de calendário criada pelo autor, com uma linha por dia, e, ao lado dela, quatro tabelas automáticas: a tabela-modelo e três tabelas locais, uma para cada coluna de data. As quatro aumentam o modelo sem necessidade, porque o calendário próprio já cumpre o papel delas. A MOD-001 aponta as quatro.

A MOD-005 aponta a própria `DimCalendar`. **Marcar como tabela de data** é a forma de dizer ao Power BI qual tabela é o calendário oficial do modelo; a marcação é exigida pelas funções clássicas de inteligência temporal do DAX, como as de acumulado do ano, e, quando ela é feita, o Power BI remove as tabelas automáticas que havia criado (Microsoft, 2025e). Como as colunas de data da `DimCalendar` ainda apontam para tabelas automáticas, a tabela comprovadamente não foi marcada, e esta é a prova que a regra usa, como descrito no Capítulo 3.

Figura 4 – Achado do P8 com a explicação gerada pela ferramenta (identificador da tabela abreviado)

Achado (regra MOD-001): A tabela oculta 'LocalDateTable_0d3564f1-…' foi gerada pelo Tempo automático de data/hora, não pelo autor do modelo.

Explicação: A tabela 'LocalDateTable_0d3564f1-…' foi gerada pelo Auto date/time, o que pode aumentar o tamanho do modelo e dificultar a propagação de filtros entre tabelas. Isso não é recomendado para modelos mais complexos ou quando uma única tabela de datas é desejável.

Recomendação: Crie uma tabela de datas manualmente usando DAX, como a função CALENDAR ou CALENDARAUTO, e conecte-a às tabelas relevantes. Isso permitirá uma propagação de filtros mais eficiente e manterá o modelo mais limpo.

Trechos citados: [1] Auto date/time guidance in Power BI Desktop › Recommendations (âncora); [3] … › Considerations; [5] Design guidance for date tables in Power BI Desktop › Use Auto date/time.

Fonte: Saída da ferramenta no projeto P8, execução de 13/10/2026.

A Figura 4 mostra um desses achados como sai da ferramenta. A explicação é correta e está ancorada: o tamanho do modelo e a impossibilidade de propagar um filtro de data a várias tabelas estão entre as considerações da página citada, e a recomendação é a que a própria página faz (Microsoft, 2025d). Na primeira execução completa, todos os 15 achados receberam explicação com citação válida, e 14 citaram a seção âncora da regra.

A leitura à mão das 15 explicações, porém, encontrou dois erros que nenhum critério automático acusou, ambos na MOD-006. Num deles, a recomendação mandava integrar numa única tabela a dimensão de produto, a de subcategoria e também a tabela de fatos de vendas; juntar fatos e dimensão é um erro de modelagem. No outro, recomendava uma tabela "normalizada" quando a fonte diz o contrário, desnormalizar. Como todos os nomes citados existiam no achado, a conferência automática não tinha o que acusar. Foram esses casos que levaram a rubrica a ter um item próprio para a correção da recomendação.

Quanto ao desempenho, a geração custou em média 8,1 segundos por achado, com pior caso de 10,5 segundos, numa GPU de 6 GB, e a auditoria completa do P8 levou 123 segundos, incluindo a carga do modelo. Duas execuções do mesmo projeto não produzem textos idênticos, apesar da temperatura 0: os totais se repetem, mas a redação e algumas citações variam, e numa das execuções pela interface 13 dos 15 achados citaram a âncora. Com o cache ligado, a segunda auditoria do P8 caiu de 165,6 segundos para 1,5 segundo, com textos idênticos aos da primeira.

5 DISCUSSÃO E AMEAÇAS À VALIDADE

Este capítulo interpreta os resultados do Capítulo 4, examina o que pode tê-los distorcido e delimita o que a ferramenta não faz. A intenção é que o leitor saiba exatamente até onde cada número vale.

5.1 O QUE OS RESULTADOS MOSTRAM

Lidos em conjunto, os resultados contam uma história coerente sobre as três camadas da ferramenta.

A **detecção** se sustenta. Seis das nove regras não emitiram nenhum falso positivo, e a precisão total, de 0,88, ficou acima do limiar de 0,7 fixado no planejamento. O único caso abaixo do limiar, a MOD-003, falhou num ponto específico e identificável: tabelas que o Power BI ou o autor criam desconectadas de propósito. Quatro dos seis erros tinham sinal estrutural no arquivo e foram corrigidos; os outros dois expõem um limite real do que se pode decidir lendo apenas a definição do modelo. Esse resultado é o que o critério de detectabilidade prometia: regras poucas, mas confiáveis, com os casos de erro explicados e não escondidos.

A **recuperação** não é o gargalo. A decisão de pôr sempre a passagem âncora da regra como primeiro trecho do contexto garantiu que o modelo de linguagem tivesse, em todos os achados, a passagem que sustenta o problema, e a leitura humana confirmou que todas as 61 explicações citaram uma passagem pertinente. A escolha do tipo de busca, que ocupou boa parte do projeto, afetou apenas os trechos complementares, e ali o defeito que mais pesou foi a construção da consulta de uma regra, não o tipo de busca.

A **geração** é o ponto fraco. Com a passagem certa nas mãos, o modelo de 7 bilhões de parâmetros escreveu uma explicação fiel em pouco mais da metade dos casos, e só pouco mais de um terço das explicações pode ser entregue sem edição. Os erros não são aleatórios: concentram-se em duas regras e repetem padrões, como o erro de tradução de “comprimir” e a leitura invertida da recomendação sobre o valor vazio. Isso tem duas leituras. A pessimista é que um modelo local desse porte ainda não redige explicações técnicas confiáveis sem supervisão. A útil é que erros sistemáticos são tratáveis: um glossário de termos na instrução, exemplos de resposta correta para as regras mais problemáticas ou um modelo maior são caminhos concretos, discutidos no Capítulo 6.

Por fim, a avaliação mostrou que a verificação automática da fidelidade, que confere números e nomes, não substitui a leitura humana: as infidelidades reais são de sentido. Esse resultado mudou a própria ferramenta, que passou a avisar o usuário de que a ausência de alerta não garante a correção do texto. A consequência prática é que a ferramenta deve ser usada como apoio a um analista, que revisa as explicações, e não como um auditor autônomo. O achado e

a recomendação padrão de cada regra, que não dependem do modelo de linguagem, são a parte em que se pode confiar sem revisão.

5.2 AMEAÇAS À VALIDADE

Uma *ameaça à validade* é qualquer fator que possa tornar uma conclusão do estudo incorreta ou menos geral do que parece. A literatura de engenharia de software experimental organiza essas ameaças em quatro tipos (Wohlin et al., 2012). A validade de *construto* pergunta se as medidas medem o que se diz que medem. A validade *interna* pergunta se o resultado se deve ao que se estudou ou a outro fator, como um viés na anotação. A validade *externa* pergunta até onde o resultado se generaliza para além dos casos estudados. A validade de *conclusão* pergunta se a análise dos dados sustenta as conclusões tiradas. As subseções seguintes examinam cada uma.

5.2.1 Validade de construto

A precisão de uma regra depende do que se aceita como violação. Para que o julgamento não fosse uma questão de gosto, o anotador julgou cada candidato contra a pergunta da passagem âncora da regra, e não contra a sua opinião sobre boas práticas. A precisão medida é, portanto, a precisão em relação ao que a documentação da Microsoft recomenda; quem discordar da documentação num caso concreto pode discordar do veredito.

A medida que escolheu o método de busca tem um viés conhecido: as consultas de avaliação foram montadas com os termos da própria regra, os mesmos que aparecem na passagem âncora. Isso facilita a tarefa e produziu o efeito teto descrito no Capítulo 4. O acerto de 1,00 entre os cinco primeiros resultados mede a recuperação dado o vocabulário da regra, e não a capacidade de responder a uma pergunta livre de um usuário.

O item “utilizável como está” condensa três julgamentos num só, e um “não” em qualquer item basta para recusar a explicação. É uma régua deliberadamente exigente: uma explicação com uma única palavra mal traduzida conta como não utilizável, embora um leitor atento ainda pudesse aproveitá-la. A proporção de 0,38 deve ser lida como a fração de explicações que dispensam qualquer edição, e não como a fração de explicações úteis.

5.2.2 Validade interna

A ameaça mais séria é o **anotador único**. O gabarito das regras e a rubrica da geração foram feitos pelo autor da ferramenta, sem um segundo anotador e, portanto, sem medida de concordância entre anotadores. O protocolo previa um segundo anotador às cegas em dois dos sete projetos; a decisão de não fazê-lo foi tomada e registrada, e a limitação é declarada aqui. Três cuidados a atenuam, sem eliminá-la. A anotação foi cega: a planilha listava todos os candidatos sem indicar quais a ferramenta marcara, e a planilha da rubrica não mostrava nenhum

dos sinais automáticos. A justificativa foi obrigatória nos vereditos negativos e indeterminados. E os julgamentos foram conferidos contra a documentação.

Ainda assim, houve **contaminação**. Antes de o gabarito ser montado, os achados das regras em P1 a P7 já tinham sido vistos, e parte deles revista, durante o desenvolvimento. Ver a saída da ferramenta antes de anotar tende a favorecê-la. O protocolo respondeu congelando as regras numa marca de versão, mas a marca foi criada no fim da anotação, e não no início; como nenhuma regra mudou no intervalo, ela marca o código efetivamente medido.

Depois da primeira medição, 12 julgamentos foram revistos com base na documentação. As revisões foram registradas, cada uma com o motivo e o julgamento original, e andaram nos dois sentidos: algumas a favor da ferramenta, como as três divisões protegidas da DAX-001, e outras contra, como a tabela de negócio sem relacionamento da MOD-003. Mesmo documentadas, revisões feitas depois de ver o resultado são uma ameaça e ficam declaradas.

As correções da MOD-003 e da PERF-001 foram motivadas pelos próprios casos de teste. Por isso o seu efeito foi relatado como análise de erros, e não como resultado; o resultado oficial é o das regras congeladas. Validar as correções exigiria anotar um projeto que as regras ainda não viram, o que não foi feito.

Há ainda uma ameaça sutil na construção do gabarito. Os candidatos que o anotador julgou foram listados a partir do mesmo analisador do `model.bim` que alimenta as regras. Um objeto que o analisador não lê fica de fora dos dois lados, e um eventual problema nele não aparece nem como acerto nem como erro.

Na recuperação, a regra de decisão sobre o tipo de busca usou todas as 70 consultas, das quais as 9 consultas por regra pesam cerca de 41% da média. Considerando só as consultas por achado, que são as que a ferramenta de fato faz, a busca densa teria vencido. A regra foi fixada antes da medição e mantida; a divergência fica registrada como uma escolha que poderia ter sido outra.

5.2.3 Validade externa

Os sete projetos avaliados são amostras públicas da Microsoft, escolhidas pela licença aberta e pela reprodutibilidade. Amostras feitas para demonstrar o produto podem ser mais limpas que modelos corporativos reais, o que reduz o número de achados e pode favorecer a precisão.

O número de casos por regra é pequeno. A MOD-001 e a MOD-007 tiveram um achado cada; a MOD-002 e a MOD-005, nenhum. Os intervalos de Wilson do Capítulo 4 mostram o quanto isso limita as conclusões por regra: para essas regras, a avaliação mostra apenas que elas não erraram nos poucos casos que encontraram. Além disso, a PERF-003 responde por 16 dos

46 achados confirmados, e a PERF-001 por metade das explicações avaliadas, de modo que os números agregados refletem sobretudo essas duas regras.

A geração foi avaliada com um único modelo de linguagem, de 7 bilhões de parâmetros, quantizado, numa única GPU de 6 GB. Os resultados não se estendem a modelos maiores, nem a outros modelos do mesmo porte. A base de documentação foi coletada em outubro de 2026; a documentação do Microsoft Learn é atualizada continuamente, e uma passagem âncora pode mudar ou sair do ar.

5.2.4 Validade de conclusão e reprodutibilidade

As proporções foram apresentadas com intervalos de confiança, e nenhuma conclusão do trabalho depende de uma diferença pequena entre duas proporções. Onde a diferença era pequena, como na escolha da busca, a decisão foi tomada por uma regra fixada antes e declarada como tal.

A reprodutibilidade tem uma limitação própria: o Ollama não é determinístico, mesmo com temperatura 0 e semente fixa. Duas execuções sobre o mesmo projeto produzem os mesmos achados e o mesmo número de explicações, mas textos e algumas citações diferentes. Para que a avaliação pudesse ser conferida, as respostas avaliadas foram guardadas em cache, e repetir a rodada reproduz exatamente os textos julgados. Os projetos P1 a P7 são públicos, e os vereditos, os resultados e o código de medição estão versionados no repositório; os textos do Microsoft Learn usados como contexto não estão, porque os termos de uso do site não permitem redistribuí-los, mas podem ser coletados de novo pelo código do projeto. O P8 não pode ser reproduzido por terceiros, e por isso nenhuma métrica depende dele.

5.3 LIMITES DE ESCOPO

Além das ameaças à validade, a ferramenta tem limites deliberados de escopo, definidos no planejamento e mantidos até o fim.

- **Formato do modelo.** A ferramenta lê só o `model.bim`, em TMSL. Projetos salvos em TMDL e arquivos PBIX precisam ser salvos antes como PBIP em TMSL. O próprio recurso de projetos do Power BI Desktop ainda está em pré-visualização (Microsoft, 2026).
- **Camada de relatório.** As páginas e os visuais não são lidos. Por isso nenhuma regra pode afirmar que uma coluna não é usada, como mostrou a rejeição da PERF-005, e problemas próprios do relatório, como o excesso de visuais numa página, não são detectados.
- **Regras de Power Query M.** O grupo de regras sobre as consultas em M saiu do produto mínimo, e nenhuma regra examina a preparação dos dados.
- **DAX dependente de contexto.** Sem um analisador sintático completo do DAX, as regras que dependem do contexto de avaliação de uma fórmula ficaram de fora, porque uma heurística nesses casos erraria com frequência.

- **Medidas em tempo de execução.** A ferramenta não mede o tamanho real das colunas nem o tempo das consultas, o que exigiria carregar o modelo com os dados; as regras de desempenho apontam riscos conhecidos, não lentidão medida.
- **Segurança em nível de linha.** A ferramenta lê as expressões de filtro dos papéis de segurança, mas não foi verificado na prática se o Power BI Desktop sempre as grava no `model.bim`.

REFERÊNCIAS

COHEN, Jacob. A coefficient of agreement for nominal scales. **Educational and Psychological Measurement**, v. 20, n. 1, p. 37–46, 1960.

CORMACK, Gordon V.; CLARKE, Charles L. A.; BÜTTCHER, Stefan. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. **Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09), Boston**, p. 758–759, 2009.

FEINSTEIN, Alvan R.; CICCHETTI, Domenic V. High agreement but low kappa: I. The problems of two paradoxes. **Journal of Clinical Epidemiology**, v. 43, n. 6, p. 543–549, 1990.

GAO, Tianyu *et al*. Enabling large language models to generate text with citations. **Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023), Singapore**, p. 6465–6488, 2023.

JI, Ziwei *et al*. Survey of hallucination in natural language generation. **ACM Computing Surveys**, v. 55, n. 12, p. 248:1–248:38, 2023.

JOHNSON, Brittany *et al*. Why don't software developers use static analysis tools to find bugs?. **Proceedings of the 35th International Conference on Software Engineering (ICSE 2013), San Francisco**, p. 672–681, 2013.

KIMBALL, Ralph; ROSS, Margy. **The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling**. 3. ed. Indianapolis: John Wiley & Sons, 2013.

LEWIS, Patrick *et al*. Retrieval-augmented generation for knowledge-intensive NLP tasks. **Advances in Neural Information Processing Systems**, v. 33, p. 9459–9474, 2020.

MICROSOFT. **DIVIDE function vs divide operator (/) in DAX**. 2021. Disponível em: <https://learn.microsoft.com/en-us/dax/best-practices/dax-divide-function-operator>. Acesso em: 8 out. 2026.

MICROSOFT. **DAX overview**. 2023a. Disponível em: <https://learn.microsoft.com/en-us/dax/dax-overview>. Acesso em: 15 out. 2026.

MICROSOFT. **Best Practice Rules**. 2023b. Disponível em: <https://github.com/microsoft/Analysis-Services/tree/master/BestPracticeRules>. Acesso em: 15 out. 2026.

MICROSOFT. **Understand star schema and the importance for Power BI**. 2024a. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/star-schema>. Acesso em: 15 out. 2026.

MICROSOFT. **Data reduction techniques for Import modeling**. 2024b. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/import-modeling-data-reduction>. Acesso em: 15 out. 2026.

MICROSOFT. **One-to-one relationship guidance**. 2024c. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/relationships-one-to-one>. Acesso em: 15 out. 2026.

MICROSOFT. **Semantic Link Labs**. 2024d. Disponível em: <https://github.com/microsoft/semantic-link-labs>. Acesso em: 15 out. 2026.

MICROSOFT. **Bi-directional relationship guidance**. 2024e. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/relationships-bidirectional-filtering>. Acesso em: 15 out. 2026.

MICROSOFT. **Semantic model modes in the Power BI service**. 2025a. Disponível em: <https://learn.microsoft.com/en-us/power-bi/connect-data/service-dataset-modes-understand>. Acesso em: 15 out. 2026.

MICROSOFT. **Data types in Power BI**. 2025b. Disponível em: <https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-data-types>. Acesso em: 15 out. 2026.

MICROSOFT. **Model relationships in Power BI Desktop**. 2025c. Disponível em: <https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-relationships-understand>. Acesso em: 15 out. 2026.

MICROSOFT. **Auto date/time guidance in Power BI Desktop**. 2025d. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/auto-date-time>. Acesso em: 15 out. 2026.

MICROSOFT. **Set and use date tables in Power BI Desktop**. 2025e. Disponível em: <https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-date-tables>. Acesso em: 15 out. 2026.

MICROSOFT. **Power BI Desktop project semantic model folder**. 2026. Disponível em: <https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-dataset>. Acesso em: 15 out. 2026.

MICROSOFT. **What is Power Query?**. 2026. Disponível em: <https://learn.microsoft.com/en-us/power-query/power-query-what-is-power-query>. Acesso em: 15 out. 2026.

OTYKIER, Daniel. **Tabular Editor 2**. *[S.d.]*. Disponível em: <https://github.com/TabularEditor/TabularEditor>. Acesso em: 15 out. 2026.

QWEN TEAM. Qwen2.5 Technical Report. **arXiv preprint arXiv:2412.15115**, 2024.

ROBERTSON, Stephen; ZARAGOZA, Hugo. The Probabilistic Relevance Framework: BM25 and Beyond. **Foundations and Trends in Information Retrieval**, v. 3, n. 4, p. 333–389, 2009.

SADOWSKI, Caitlin *et al*. Lessons from building static analysis tools at Google. **Communications of the ACM**, v. 61, n. 4, p. 58–66, 2018.

TABULAR EDITOR. **Best Practice Analyzer**. *[S.d.]*. Disponível em: <https://docs.tabulareditor.com/te2/Best-Practice-Analyzer.html>. Acesso em: 15 out. 2026.

THAKUR, Nandan *et al*. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. **arXiv preprint arXiv:2104.08663**, 2021.

VAN GULCK, Nat. **Fab Inspector (PBI Inspector V2)**. *[S.d.]*. Disponível em: <https://github.com/NatVanG/PBI-InspectorV2>. Acesso em: 15 out. 2026.

VASWANI, Ashish *et al*. Attention is all you need. **Advances in Neural Information Processing Systems**, v. 30, p. 5998–6008, 2017.

WILSON, Edwin B. Probable inference, the law of succession, and statistical inference. **Journal of the American Statistical Association**, v. 22, n. 158, p. 209–212, 1927.

WINTERS, Titus; MANSHRECK, Tom; WRIGHT, Hyrum. **Software Engineering at Google: Lessons Learned from Programming Over Time**. Sebastopol: O'Reilly Media, 2020.

WOHLIN, Claes *et al*. **Experimentation in Software Engineering**. Berlin, Heidelberg: Springer, 2012.

XIAO, Shitao *et al*. C-Pack: Packaged resources to advance general Chinese embedding. **arXiv preprint arXiv:2309.07597**, 2023.